



Fatih DEMİRBİLEK 2526234

Ali Buğra Şahin 2526663

Ali Hikmet Sağaltıcı 2585313

İçindekiler

Project Description	3
Data Requirements	3
Data Entry	6
Data Update/Deletion.....	7
Data Queries	7
EER	8
Set of Relations	9
Functional Dependencies	10
Normalisations.....	12
Finalised Normalised Relations	16
Denormalisation	18
Queries.....	18
Discussion.....	28
Workload & Indexes.....	29
Changes and Modifications to the Database	30
Implementation Details.....	37
Application.....	37

Project Description

This web application will be designed for a cryptocurrency trading platform with features such as cryptocurrency trading, user wallet management and customer support. Each User will have a dedicated Wallet that can hold multiple currencies. The System records all Transactions for adding or withdrawing funds to a Wallet, and each transaction requires approval from a Customer Service Agent. Users can create Orders on different Markets. Markets are different trading pairs on cryptocurrencies. An Order Book tracks open or partially open orders that can be matched to form completed Trades which is a history of orders. Price History stores history of pricing based on finalized trades to inform trading decisions. Additionally, the application has an Issues management system. Users can report problems that Customer Service Agents can review and solve. The project design emphasizes data integrity with storing wallets separately, allows faster trading by keeping track of open and close orders separately and enhances user experience by mapping clear relationships among User, Wallet, Transaction, Currency, Market, Price History, Order, Trade, Order Book, Customer Service Agent, Issue.

Data Requirements

Wallet

Wallets are created alongside a new user. They're separate, but tied to the specific user they're created for and defined by their unique ID. Wallet also holds the balance and status information to view. With this, Users can check their balance and their wallet status, and act accordingly.

User

Users are uniquely defined by their ID number, meaning any one person can have one user account. User also carry other information such as name, surname, address, email, phone number and KYC. KYC is a requirement for centralised exchanges to verify user identities and holds National ID, KYC status and verify time given from KYC system. Users can also set a unique password to protect their account from unauthorised Access. Every user has their own wallet and both depend on each other; upon user creation a unique wallet is created for that user.

Transaction

Transactions are order requests that users give to the market; such as buy crypto or convert USD to TRY. Every transaction is defined by their unique transaction ID. Transactions are made by the user; they also hold transaction information such as currency and transaction type, the transaction amount, status and holds the information of when transaction request was made and when it was approved (or disapproved).

Issue

Issues are reported by users and they're used to notify Customer Service Agents about possible problems in the database, program or any current or old transaction. Each issue is uniquely defined by their ID. Issue status, description, creation and resolving time and issue type informations are also held. With this, issues are efficiently categorised, prioritised and solved.

Customer Service Agent

Main purposes of customer service agents are resolving issues reported by users and confirming (or declining) the transaction requests. They're defined by their ID, and each holds name, surname, employment time, email and phone information.

Currency

The Currency entity represents all the currencies that are handled within the cryptocurrency exchange system. Each currency is assigned a unique ID and has a descriptive Name (such as Bitcoin, Ethereum, or US Dollar). Each currency also has a Symbol that acts as an abbreviation (such as BTC, ETH, or USD) for use throughout the system. There is also a Status for every currency, which indicates if it is presently available or active in the exchange (active, inactive, or suspended). Two distinct subclasses exist for currencies: Crypto Currency and Fiat Currency, with their own specialized attributes.

The Crypto Currency subclass is utilized to define digital currencies based on blockchain technology, such as Bitcoin or Ethereum. Important financial data like Market Cap, the overall market value of the currency, and Trading Volume, the overall trading volume of the currency over a span of time, are monitored. In addition, characteristics like Circulation Supply, the volume of currency present and actively being traded in the marketplace, and the Total Supply, the number of available coins, are given. The organization also maintains the Max Supply, which is the absolute maximum potential number of coins that will ever be in circulation. Historical performance metrics, such as All Time High and All Time Low, are maintained to track the price highs and lows of the currency over its lifetime.

The Fiat Currency subclass specifies traditional currencies that are controlled and issued by governmental or central financial authorities, such as Euros or US Dollars. Each fiat currency includes details on its Origin, which specifies the country or governing body that issues and controls the currency, making it easy for users to distinguish between available fiat options on the platform.

Market

The Market entity specifies individual trading markets provided within the cryptocurrency exchange system. Each market facilitates trading activity between exactly two different currencies. Each market in the system is referenced by a unique market identifier (Market ID) and includes a status of the market being active, inactive, or in maintenance. It also includes a Creation Time attribute, which specifies the exact time when the market was initially created. A Market entity is associated with the Currency entity, defining precisely which currencies are being exchanged, and is associated with precisely one Order Book, which classifies all buy and sell orders. Each market also records its past price data via the associated Price History entity.

Price History

The Price History entity contains past price information that relates specifically to each Market in the cryptocurrency exchange system. Each Price History record holds comprehensive details about the price movement of a particular currency pair for specific time intervals. These are the opening price, closing price, highest price (High Price), lowest price (Low Price), and time interval of these price records. This entity allows user to view price history of each currency and help them in their order decisions. Each price history entry is identifiable uniquely and relates directly to a single Market entity, allowing users and analysts to track market performance over time intervals.

Order Book

The Order Book entity keeps the list of all the buy and sell orders currently open and waiting to be fulfilled for a particular Market. There is one Order Book per market in the exchange system. Orders in order book can be not fulfilled, partially fulfilled and whenever it's completely fulfilled it will be removed from the Order Book. The Order Book is designated by a Book ID uniquely and maintains an Update Time to indicate the recent updates or changes made in it. The Order Book entity stores and maintains all the orders placed (via the Order entity), indicating their status, prices, and quantities ordered, with real-time visibility and enabling trade matching effectively.

Order

The Order entity represents individual trading orders placed by users in the crypto exchange system. Each order contains the essential information, including a unique identifier (Order ID), the type of transaction (buy or sell), the quantity of currency to exchange, the preferred price for the transaction, the creation timestamp when the order was placed, and optionally, the close timestamp for when the order was filled or cancelled. Each order is knowingly an element of a single Order Book, making it easier for the system and traders to classify and match buy and sell orders efficiently. Orders become Trades when they are matched and executed successfully in the market.

Trade

The Trade entity captures executed transactions that come about when a buy and a sell order are matched successfully within an Order Book. Every trade is given a unique identification, logging information like the transaction volume, the precise transaction price, and the timestamp of when the trade occurred. Trades are explicitly linked to particular orders, making it clear which buy and sell orders were paired to create the transaction that was completed. Therefore, each executed trade leaves a clear historical record for the users and the exchange system, promoting accountability, compliance, and audibility of transactions.

Data Entry

- ❖ Wallet: Enter/update wallet details (such as balance: 2.35 BTC, status: active) ID is generated.
- ❖ User: Enter the user details (such as new user: name:Ali Hikmet, surname: Sağaltıcı, email: sagalticialihikmet@gmail.com, phone: +90533*, address: Kalkanlı/Güzelyurt, password: password, attachments for KYC verification). ID is generated.
- ❖ Transaction: Enter the transaction details (type: sell, currency: BTC, amount: 0.003), ID, request time, approval time are generated, status is on hold as default and updated per customer agent actions and buyer/seller situation.
- ❖ Issue: Enter issue details (type: wallet problem, description: "I'm unable to view my wallet balance"). ID, creation time are generated. Status is sent as default and resolve time is none as default, both updated per customer agent actions.
- ❖ Customer Service Agent: Enter agent details (name: Fatih, email: fatih*@hotmail.com, phone: +905**, 15/03/2025). ID is generated.
- ❖ Currency: Enter currency details (such as for a fiat currency: name: US Dollars, symbol: \$, status: active), ID is generated.
- ❖ Crypto Currency: Enter currency details (such as for BTC: rank: 1, market cap: 1,63T USD, trading volume: 421K \$, circulation supply: 15k, total supply: 15T \$, max supply: 2T \$, all time high: 105.000\$, all time low: 1\$)
- ❖ Fiat Currency: enter currency details (such as for USD: origin: USA)
- ❖ Order: Enter order details (such as type: BTC, price: 100.000\$, quantity: 0,0003). Creation time is generated by time of order, close time is generated by order fulfillment or cancellation time. ID is generated randomly.
- ❖ Market: Enter market details (such as name: Crypto, status: active). Creation time is generated by time of market creation, ID is generated.
- ❖ Trade: Enter trade details (such as price: 12,50\$, quantity: 9). trade time is generated at trade completion. ID is generated.
- ❖ Order Book: order book details are filled with open orders. As long as an order is not completed or canceled, it will be present in an order book and update time will be updated whenever any action is observed in the order, starting from creation time. ID is taken from order ID. Because of these reasons, no data entry is needed.
- ❖ Price history: Price history is generated by daily/weekly/monthly price changes. Price ID is derived from currency ID, time interval is set by user to show time interval of crypto/fiat

price and open price/close price and high/low price is generated by opening, closing and highest/lowest prices of said item in that day.

Data Update/Deletion

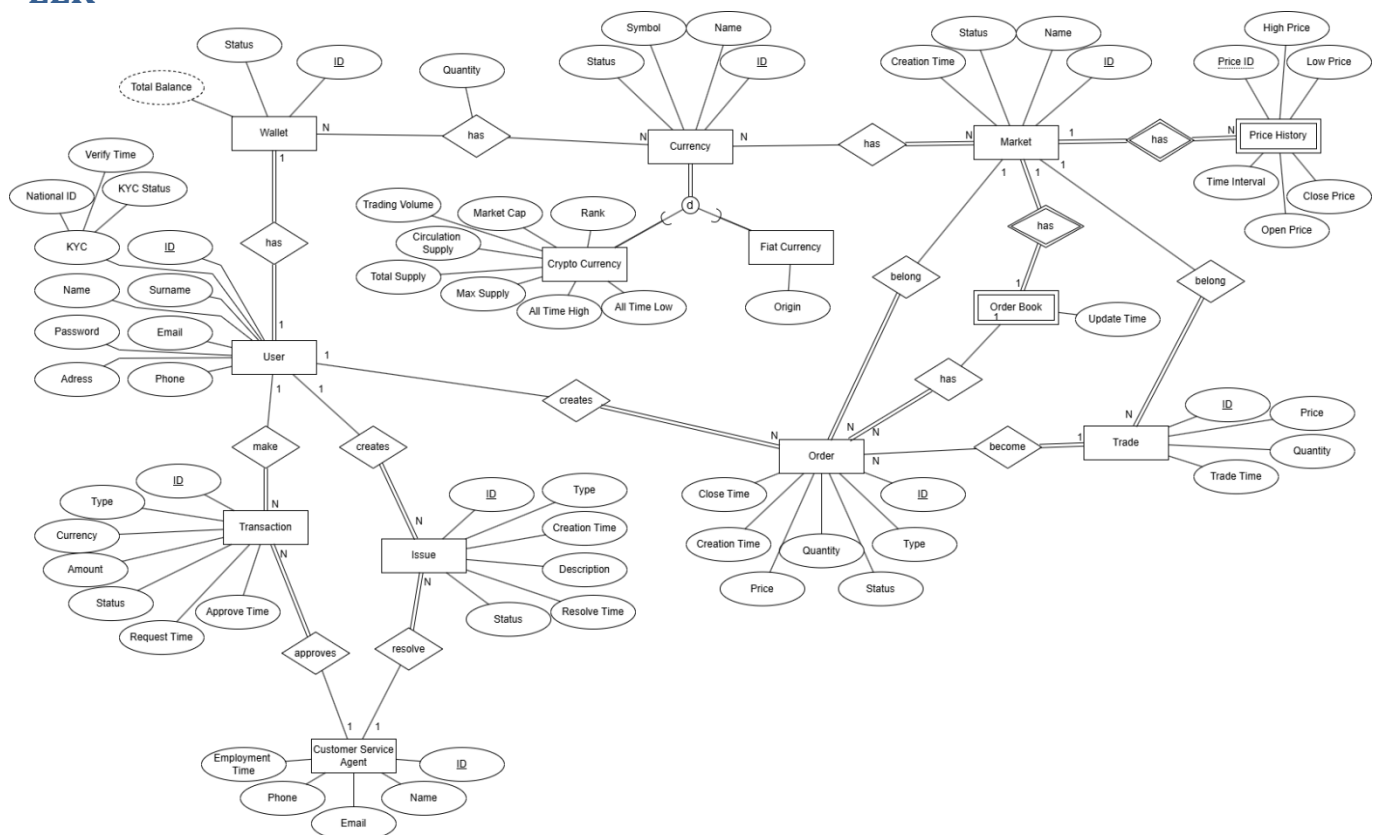
- Update/delete Currency details: Agents can update or delete details associated with a currency, such as changing its status (active, inactive, suspended), updating its symbol, or modifying descriptive information.
- Update/delete order details within an Order Book. For instance, system can update the quantity, price, or status of existing orders or delete orders that have expired or were placed incorrectly by users.
- Update/delete trade details resulting from transactions within a Market. This could involve correcting mistakenly entered transaction quantities or prices, or deleting trade records if needed due to transaction errors or cancellations.
- Update/delete wallet details: Update or delete details related to a user's wallet, such as modifying balance information or changing wallet status (e.g., active, suspended, closed).
- Update/delete user details: Agents can update or delete personal details associated with users, including name, surname, address, email, phone number, and KYC status. User accounts may also be disabled, deleted, or reactivated by an agent.
- Update transaction details: System can modify transaction details, such as changing transaction status (approved, pending, or rejected), adjusting transaction amounts, or correcting associated currency and transaction type details.
- Update issue details: Agents can update issue-related information, including issue status (open, resolved, pending), description, issue type, or closing of reported issues when resolved or deemed invalid.
- Update/delete customer service agent details: Update or delete personal and employment details of customer service agents, such as their name, surname, email, phone number, or employment dates.

Data Queries

- List all currencies available in the system.
- List currencies based on their current status (active/inactive).
- Select details of a specific currency by its symbol or name.
- List all cryptocurrencies sorted by market cap.
- List cryptocurrencies that have reached a new all-time high or low.
- List all fiat currencies available in the system.
- Select the origin country information for a specific fiat currency.
- Select active markets involving a specific currency.
- Select market details by market ID or currency pairs involved.
- Select price history records for a specific market.

- List price history for a currency pair within a certain time range.
- Select the order book details for a specific market.
- List order books updated within a certain period.
- List all orders placed by users within a certain market.
- Select orders filtered by status (completed, pending, cancelled).
- Select wallet details by associated user ID (for agent).
- List wallets based on their current status (active, suspended, closed).
- Select a specific registered user and list their basic information (for agent).
- List users by their KYC verification status (for agent).
- List all active customer service agents.
- List transactions approved or declined by a particular customer service agent.
- List all reported issues sorted by creation date (for agent).
- List issues by their current status (open, pending, resolved) (for agent).
- Select detailed information for a specific issue by its ID.
- List all orders placed by users.
- Select details of a specific order by order ID.
- List orders filtered by their status (completed, pending, cancelled).
- Select details of trades executed within a certain date range (for agent).

EER



Assumptions:

- ✓ Data for CryptoCurrency entity is gathered from online resources and other exchanges. They are not derived from only our exchanges data.
- ✓ Order Book only stores and maintains open orders.
- ✓ If Orders are completed, they become Trade and they will be deleted from the Order Book. However, Order is still stored. Normally Order Book is maintained in-memory throughout the application however we decided to store it in the database.
- ✓ Price History will be used for graphing. It could be derived from other entities and attributes but for faster data access it is also stored.
- ✓ Currencies may not be open for trade hence they don't require to be in a market as trading pairs.

Set of Relations

User									
<u>User ID</u>	Email	Name	Surname	Phone	Address	Password	Verify Time	National ID	KYC Status

Wallet			
<u>Wallet ID</u>	Status	Total Balance	FK:User:User ID

Currency			
<u>Currency ID</u>	Name	Symbol	Status

Crypto_Currency									
FK:Currency:Currency ID	Rank	Market Cap	Trading Volume	Circulation Supply	Total Supply	Max Supply	ATH	ATL	

Fiat_Currency	
FK:Currency:Currency ID	Origin

Market			
<u>Market ID</u>	Name	Status	Creation Time

Trade				
<u>Trade ID</u>	Price	Quantity	Trade Time	FK:Market:Market ID

Order										
<u>Order ID</u>	Type	Status	Quantity	Price	Creation Time	Close Time	FK:User:User ID	FK:Market:Market ID	FK:Trade:Trade ID	FK:Order Book:Market ID

Issue								
<u>Issue ID</u>	Type	Creation Time	Description	Resolve Time	Status	FK:User:User ID	FK:Customer_Service_Agent:Agent ID	

Transaction									
<u>Transaction ID</u>	Type	Currency	Amount	Status	Request Time	Aprove Time	FK:User:User ID	FK:Customer_Service_Agent:Agent ID	

Customer_Service_Agent				
<u>Agent ID</u>	Name	Email	Phone	Employment Time

Price_History						
<u>Price ID</u>	<u>FK:Market ID</u>	High Price	Low Price	Open Price	Close Price	Time Interval

Order_Book	
<u>Market ID</u>	Update Time

Wallet_has_Currency		
<u>Wallet ID</u>	<u>Currency ID</u>	Quantity

Currency_has_Market	
<u>Currency ID</u>	<u>Market ID</u>

Functional Dependencies

User

User ID->Email,Name,Surname,Phone,Adress>Password, Verify Time, National ID, KYC Status

Email ->User ID,Name,Surname,Phone,Adress>Password, Verify Time, National ID, KYC Status

National ID ->User ID,Email,Name,Surname,Phone,Adress>Password, Verify Time,KYC Status

Note: Verify Time attribute can be null. Technically Verify Time attribute if exists determines the Status attribute so it might be considered as an FD. However, after discussion we decided not to continue with this FD because Status can be Open, Partially Open and Closed. Verify Time only determines if Status is Closed or not others.

Wallet

Wallet ID-> Status>Total Balance,User ID

User ID->Wallet ID,Status,Total Balance

Note: Since User and Wallet relation is one-to-one User ID is unique for Wallet Table

Currency

Currency ID->Name,Symbol,Status

Symbol->Name

Note: Symbols for Crypto Currencies are not regulated and generally exchanges are self regulated. If someone want to create a new coin with an existing symbol markets allow that however in our exchange we also want to regulate it so our Crypto Symbols determine the name of the crypto. Thats why we have the FD with Symbol and Name.

Crypto Currency

Currency ID->Rank,Market Cap,Trading Volume,Circulation,Total Supply,Max Supply,ATH,ATL

Fiat Currency

Currency ID->Origin

Market

Market ID->Name,Status,Creation Time

Trade

Trade ID->Price,Quantity,Trade Time,Market ID

Order

Order ID->Type,Status,Quantity,Price,Creation Time, Close Time, User ID, Market ID, Trade ID

Issue

Issue ID->Type,Creation Time,Description,Resolve Time, Status, User ID, Agent ID

Note: Resolve Time attribute can be null until the issue is resolved.

Transaction

Transaction ID->Type,Currency,Amount,Status,Request Time,Approve Time,User ID, Agent ID

Customer Service Agent

Agent ID->Name,Email,Phone,Employment Time

Email->ID,Name,Phone,Employment Time

Price History

Price ID, Market ID -> High Price, Low Price, Open Price, Close Price, Time Interval

Note: Close price attribute will hold latest price until end of the day; at the end of the day it will stay as the last price of currency of that day.

Order Book

Market ID -> Update Time

Note: Update Time attribute can be null until any kind of transaction (semi or fully completed) has occurred.

Wallet_has_currency

Wallet ID, Currency ID -> Quantity

Currency_has_Market

None

Normalisations

USER TABLE NORMALISATION

User									
<u>User ID</u>	Email	Name	Surname	Phone	Adress	Password	Verify Time	National ID	KYC Status

1NF

2NF

3NF

BCNF

WALLET TABLE NORMALISATION

Wallet			
<u>Wallet ID</u>	Status	Total Balance	FK:User:User ID

1NF

2NF

3NF

BCNF

CURRENCY TABLE NORMALISATION

Currency			
<u>Currency ID</u>	Name	Symbol	Status

1NF

2NF

3NF

Currency		
<u>Currency ID</u>	FK:Currency_Symbol:Symbol	Status

Currency_Symbol	
<u>Symbol</u>	Name

1NF

2NF

3NF

BCNF

CRYPTO CURRENCY TABLE NORMALISATION

Crypto_Currency								
FK:Currency:Currency ID	Rank	Market Cap	Trading Volume	Circulation Supply	Total Supply	Max Supply	ATH	ATL

1NF
2NF
3NF
BCNF

FIAT CURRENCY TABLE NORMALISATION

Fiat_Currency	
FK:Currency:Currency ID	Origin

1NF
2NF
3NF
BCNF

MARKET TABLE NORMALISATION

Market			
Market ID	Name	Status	Creation Time

1NF
2NF
3NF
BCNF

TRADE TABLE NORMALISATION

Trade				
Trade ID	Price	Quantity	Trade Time	FK:Market:Market ID

1NF
2NF
3NF
BCNF

ORDER TABLE NORMALISATION

Order										
<u>Order ID</u>	Type	Status	Quantity	Price	Creation Time	Close Time	FK:User:User ID	FK:Market:Market ID	FK:Trade:Trade ID	Book: Market ID

1NF
2NF
3NF
BCNF

ISSUE TABLE NORMALISATION

Issue								
<u>Issue ID</u>	Type	Creation Time	Description	Resolve Time	Status	FK:User:User ID	FK:Customer_Service_Agent:Agent ID	

1NF
2NF
3NF
BCNF

TRANSACTION TABLE NORMALISATION

Transaction								
<u>Transaction ID</u>	Type	Currency	Amount	Status	Request Time	Aprove Time	FK:User:User ID	FK:Customer_Service_Agent:Agent ID

1NF
2NF
3NF
BCNF

CUSTOMER SERVICE AGENT TABLE NORMALISATION

Customer_Service_Agent				
<u>Agent ID</u>	Name	Email	Phone	Employment Time

1NF
2NF
3NF
BCNF

PRICE HISTORY NORMALISATION

Price_History						
Price ID	FK:Market ID	High Price	Low Price	Open Price	Close Price	Time Interval

1NF
2NF
3NF
BCNF

ORDER BOOK TABLE NORMALISATION

Order_Book	
Market ID	Update Time

1NF
2NF
3NF
BCNF

WALLET HAS CURRENCY TABLE NORMALISATION

Wallet_has_Currency		
Wallet ID	Currency ID	Quantity

1NF
2NF
3NF
BCNF

CURRENCY HAS MARKET TABLE NORMALISATION

Currency_has_Market	
Currency ID	Market ID

1NF
2NF
3NF
BCNF

Finalised Normalised Relations

User									
<u>User ID</u>	Email	Name	Surname	Phone	Adress	Password	Verify Time	National ID	KYC Status

Wallet			
<u>Wallet ID</u>	Status	Total Balance	FK:User:User ID

Currency		
<u>Currency ID</u>	FK:Currency_Symbol:Symbol	Status

Currency_Symbol	
<u>Symbol</u>	Name

Crypto_Currency									
FK:Currency:Currency ID	Rank	Market Cap	Trading Volume	Circulation Supply	Total Supply	Max Suppply	ATH	ATL	

Fiat_Currency	
FK:Currency:Currency ID	Origin

Market			
<u>Market ID</u>	Name	Status	Creation Time

Trade				
<u>Trade ID</u>	Price	Quantity	Trade Time	FK:Market:Market ID

Order										
<u>Order ID</u>	Type	Status	Quantity	Price	Creation Time	Close Time	FK:User:User ID	FK:Market:Market ID	FK:Trade:Trade ID	FK:Order Book:Market ID

Issue								
<u>Issue ID</u>	Type	Creation Time	Description	Resolve Time	Status	FK:User:User ID	FK:Customer_Service_Agent:Agent ID	

Transaction									
<u>Transaction ID</u>	Type	Currency	Amount	Status	Request Time	Aprove Time	FK:User:User ID	FK:Customer_Service_Agent:Agent ID	

Customer_Service_Agent					
<u>Agent ID</u>	Name	Email	Phone	Employment Time	

Price_History						
<u>Price ID</u>	FK:Market ID	High Price	Low Price	Open Price	Close Price	Time Interval

Order_Book	
<u>Market ID</u>	Update Time

Wallet_has_Currency		
<u>Wallet ID</u>	<u>Currency ID</u>	Quantity

Currency_has_Market	
<u>Currency ID</u>	<u>Market ID</u>

Denormalisation

In our initial design, we used fields like `verify_time` or `close_time` to directly show the status of a record. For example, if `verify_time` existed, the status was automatically considered "verified." This acted like a denormalized shortcut and we even defined FDs based on that.

However, later we expanded the status field to include more states. At that point, having status depend on timestamps didn't make sense anymore, so we removed those FDs and treated status as a separate, normalized attribute. The reason behind this change was whenever a "verify time" or a similar attribute was not null, "status" attribute used to change to verified, but whenever we normalised the tables and separated the status, the other statuses such as pending or rejected didn't reflect or update the attributes accordingly. This change made the model much easier to work on and manage, even if it meant adding a bit more logic elsewhere.

Queries

Create Queries

```
CREATE TABLE "User"
(
    "User ID"      INTEGER PRIMARY KEY,
    "Email"        VARCHAR(100) NOT NULL,
    "Name"         VARCHAR(60)  NOT NULL,
    "Surname"      VARCHAR(60)  NOT NULL,
    "Phone"        VARCHAR(20),
    "Address"      VARCHAR,
    "Password"     VARCHAR      NOT NULL,
    "Verify Time"  TIMESTAMP,
    "National ID"  VARCHAR(20),
    "KYC Status"   VARCHAR(30)
);

CREATE TABLE "Customer_Service_Agent"
(
    "Agent ID"      INTEGER PRIMARY KEY,
    "Name"          VARCHAR(60)  NOT NULL,
    "Email"         VARCHAR(100) NOT NULL,
    "Phone"         VARCHAR(20),
    "Employment Time" TIMESTAMP NOT NULL
);

CREATE TABLE "Wallet"
(
    "Wallet ID"     INTEGER PRIMARY KEY,
    "Status"        VARCHAR(30),
    "Total Balance" DOUBLE PRECISION,
    "User ID"       INTEGER NOT NULL
        REFERENCES "User" ("User ID")
        ON DELETE CASCADE
);

CREATE TABLE "Currency_Symbol"
(
    "Symbol"        VARCHAR(10) PRIMARY KEY,
    "Name"          VARCHAR(40) NOT NULL
);

CREATE TABLE "Currency"
(
    "Currency ID"   INTEGER PRIMARY KEY,
    "Name"          VARCHAR(40) NOT NULL,
    "Symbol"        VARCHAR(10) NOT NULL
        REFERENCES "Currency_Symbol" ("Symbol")
        ON DELETE CASCADE,
    "Status"        VARCHAR(20)
);

CREATE TABLE "Crypto_Currency"
(
    "Currency ID"   INTEGER PRIMARY KEY
        REFERENCES "Currency" ("Currency ID")
        ON DELETE CASCADE,
    "Rank"          INTEGER,
```

```

    "Market Cap"          BIGINT,
    "Trading Volume"      BIGINT,
    "Circulation Supply"  BIGINT,
    "Total Supply"        BIGINT,
    "Max Supply"          BIGINT,
    "ATH"                 DOUBLE PRECISION,
    "ATL"                 DOUBLE PRECISION
);

CREATE TABLE "Fiat_Currency"
(
    "Currency ID" INTEGER PRIMARY KEY
        REFERENCES "Currency" ("Currency ID")
        ON DELETE CASCADE,
    "Origin"       VARCHAR(60)
);

CREATE TABLE "Market"
(
    "Market ID"   INTEGER PRIMARY KEY,
    "Name"        VARCHAR(40) NOT NULL,
    "Status"      VARCHAR(20),
    "Creation Time" TIMESTAMP NOT NULL
);

CREATE TABLE "Order_Book"
(
    "Market ID"   INTEGER PRIMARY KEY
        REFERENCES "Market" ("Market ID")
        ON DELETE CASCADE,
    "Update Time" TIMESTAMP
);

CREATE TABLE "Price_History"
(
    "Price ID"     INTEGER,
    "Market ID"    INTEGER,
    "High Price"   DOUBLE PRECISION,
    "Low Price"    DOUBLE PRECISION,
    "Open Price"   DOUBLE PRECISION,
    "Close Price"  DOUBLE PRECISION,
    "Open Time"    TIMESTAMP,
    "Close Time"   TIMESTAMP,
    PRIMARY KEY ("Price ID", "Market ID"),
    FOREIGN KEY ("Market ID")
        REFERENCES "Market" ("Market ID")
        ON DELETE CASCADE
);

CREATE TABLE "Trade"
(
    "Trade ID"     INTEGER PRIMARY KEY,
    "Price"        DOUBLE PRECISION,
    "Quantity"     DOUBLE PRECISION,
    "Trade Time"   TIMESTAMP,
    "Market ID"    INTEGER NOT NULL
        REFERENCES "Market" ("Market ID")
        ON DELETE CASCADE
);

CREATE TABLE "Order"
(
    "Order ID"     INTEGER PRIMARY KEY,
    "Type"         VARCHAR(10),
    "Status"       VARCHAR(20),
    "Quantity"     DOUBLE PRECISION,
    "Price"        DOUBLE PRECISION,
    "Creation Time" TIMESTAMP,
    "Close Time"   TIMESTAMP,
    "User ID"      INTEGER NOT NULL
        REFERENCES "User" ("User ID")
        ON DELETE CASCADE,
    "Market ID"    INTEGER NOT NULL
        REFERENCES "Market" ("Market ID")
        ON DELETE CASCADE,
    "Trade ID"     INTEGER NOT NULL
        REFERENCES "Trade" ("Trade ID")
        ON DELETE CASCADE,
    "Market ID (Book)" INTEGER NOT NULL
        REFERENCES "Order_Book" ("Market ID")
        ON DELETE CASCADE
);

```

```

);

CREATE TABLE "Issue"
(
    "Issue ID"      INTEGER PRIMARY KEY,
    "Type"          VARCHAR(30),
    "Creation Time"  TIMESTAMP,
    "Description"    VARCHAR,
    "Resolve Time"   TIMESTAMP,
    "Status"         VARCHAR(20),
    "User ID"        INTEGER NOT NULL
        REFERENCES "User" ("User ID")
        ON DELETE CASCADE,
    "Agent ID"       INTEGER NOT NULL
        REFERENCES "Customer_Service_Agent" ("Agent ID")
        ON DELETE CASCADE
);

CREATE TABLE "Transaction"
(
    "Transaction ID" INTEGER PRIMARY KEY,
    "Type"           VARCHAR(20),
    "Currency"       VARCHAR(10),
    "Amount"         DOUBLE PRECISION,
    "Status"         VARCHAR(20),
    "Request Time"   TIMESTAMP,
    "Approve Time"   TIMESTAMP,
    "User ID"        INTEGER NOT NULL
        REFERENCES "User" ("User ID")
        ON DELETE CASCADE,
    "Agent ID"       INTEGER NOT NULL
        REFERENCES "Customer_Service_Agent" ("Agent ID")
        ON DELETE CASCADE
);

CREATE TABLE "Wallet_has_Currency"
(
    "Wallet ID"  INTEGER
        REFERENCES "Wallet" ("Wallet ID")
        ON DELETE CASCADE,
    "Currency ID" INTEGER
        REFERENCES "Currency" ("Currency ID")
        ON DELETE CASCADE,
    "Quantity"   DOUBLE PRECISION,
    PRIMARY KEY ("Wallet ID", "Currency ID")
);

CREATE TABLE "Currency_has_Market"
(
    "Currency ID" INTEGER
        REFERENCES "Currency" ("Currency ID")
        ON DELETE CASCADE,
    "Market ID"   INTEGER
        REFERENCES "Market" ("Market ID")
        ON DELETE CASCADE,
    PRIMARY KEY ("Currency ID", "Market ID")
);

```


Insert Queries

```
INSERT INTO "User" ("User ID", "Email", "Name", "Surname", "Phone", "Adress", "Password",
                    "Verify Time", "National ID", "KYC Status")
VALUES (1, 'aliye@gmail.com', 'Aliye', 'Öztürk', '+905311111111', 'Ankara', '4nk4r4_06',
        '2025-04-01 09:00', '12345678901', 'APPROVED'),
(2, 'baris_234@gmail.com', 'Baris', 'Yılmaz', '+905322222222', 'İstanbul', 'r2d2a3',
        NULL, NULL, 'PENDING'),
(3, 'canan_34@gmail.com', 'Canan', 'Demir', '+905333333333', 'İzmir', 'Boyozbest1',
        NULL, NULL, 'PENDING'),
(4, 'canberk@gmail.com', 'Canberk', 'Kaya', '+905344444444', 'Bursa', 'Canberking01',
        NULL, NULL, NULL),
(5, 'fatih01@gmail.com', 'Fatih', 'Demirbilek', '+905355555555', 'Adana', 'I_lovedb',
        '2025-04-05 12:00', '52345678901', 'APPROVED'),
(6, 'fransizasker2@gmail.com', 'Firat', 'Çelik', '+905366666666', 'Konya', 'm4a1_s',
        NULL, NULL, 'REJECTED'),
(7, 'güler_ayd@gmail.com', 'Güler', 'Aydın', '+905377777777', 'Eskişehir', 'OracleBest',
        NULL, NULL, 'PENDING'),
(8, 'Polat_alimdar@gmail.com', 'Alemdar', 'Polat', '+905388888888', 'Trabzon', 'Cantik_63',
        NULL, NULL, 'REJECTED'),
(9, 'ilhan21@gmail.com', 'Ilhan', 'İnan', '+905399999999', 'Samsun', 'Bandirma_vapurum',
        '2025-04-09 15:00', '92345678901', 'APPROVED'),
(10, 'Ali_dayi@gmail.com', 'Ali', 'Buğra', '+905300000000', 'Antalya', '351wasBetter',
        NULL, NULL, 'PENDING');
```

```
INSERT INTO "Customer_Service_Agent"
("Agent ID", "Name", "Email", "Phone", "Employment Time")
VALUES (1, 'Ahmet Akar', 'a.akar@cryptonit.com', '+903121001001', '2024-01-10'),
(2, 'Büşra Bilgin', 'b.bilgin@cryptonit.com', '+903121001002', '2024-01-11'),
(3, 'Cem Çetin', 'c.cetin@cryptonit.com', '+903121001003', '2024-01-12'),
(4, 'Deniz Duman', 'd.duman@cryptonit.com', '+903121001004', '2024-01-13'),
(5, 'Elif Er', 'e.er@cryptonit.com', '+903121001005', '2024-01-14'),
(6, 'Furkan Fırat', 'f.firat@cryptonit.com', '+903121001006', '2024-01-15'),
(7, 'Gökçe Güler', 'g.guler@cryptonit.com', '+903121001007', '2024-01-16'),
(8, 'Hakan Hız', 'h.hiz@cryptonit.com', '+903121001008', '2024-01-17'),
(9, 'İpek Ilgaz', 'i.ilgaz@cryptonit.com', '+903121001009', '2024-01-18'),
(10, 'Kemal Kaya', 'k.kaya@cryptonit.com', '+903121001010', '2024-01-19');
```

```
INSERT INTO "Wallet" ("Wallet ID", "Status", "Total Balance", "User ID")
VALUES (1, 'ACTIVE', 15000.00, 1),
(2, 'ACTIVE', 7500.00, 2),
(3, 'SUSPENDED', 0.00, 3),
(4, 'ACTIVE', 12000.00, 4),
(5, 'ACTIVE', 5500.00, 5),
(6, 'ACTIVE', 97557600.00, 6),
(7, 'ACTIVE', 31000.00, 7),
(8, 'ACTIVE', 10000000.00, 8),
(9, 'ACTIVE', 88000.00, 9),
(10, 'CLOSED', 0.00, 10);
```

```
INSERT INTO "Currency_Symbol" ("Symbol", "Name")
VALUES ('BTC', 'Bitcoin'),
('ETH', 'Ethereum'),
('USDT', 'Tether USD'),
('USD', 'US Dollar'),
('EUR', 'Euro'),
('ADA', 'Cardano'),
('XRP', 'Ripple'),
('BNB', 'Binance Coin'),
('SOL', 'Solana'),
('DOT', 'Polkadot'),
('JPY', 'Japanese Yen'),
('GBP', 'British Pound'),
('TRY', 'Turkish Lira'),
('CHF', 'Swiss Franc');
```

```
INSERT INTO "Currency"
("Currency ID", "Name", "Symbol", "Status")
VALUES (1, 'Bitcoin', 'BTC', 'ACTIVE'),
(2, 'Ethereum', 'ETH', 'ACTIVE'),
(3, 'Tether USD', 'USDT', 'ACTIVE'),
(4, 'US Dollar', 'USD', 'ACTIVE'),
(5, 'Euro', 'EUR', 'ACTIVE'),
(6, 'Cardano', 'ADA', 'ACTIVE'),
(7, 'Ripple', 'XRP', 'ACTIVE'),
(8, 'Binance Coin', 'BNB', 'ACTIVE'),
(9, 'Solana', 'SOL', 'ACTIVE'),
(10, 'Polkadot', 'DOT', 'ACTIVE'),
(11, 'USD', 'USD', 'ACTIVE'),
(12, 'EUR', 'EUR', 'ACTIVE'),
(13, 'JPY', 'JPY', 'ACTIVE');
```

```
(14, 'GBP', 'GBP', 'ACTIVE'),
(15, 'TRY', 'TRY', 'ACTIVE'),
(16, 'CHF', 'CHF', 'ACTIVE');
```

```
INSERT INTO "Crypto_Currency"
("Currency ID", "Rank", "Market Cap", "Trading Volume", "Circulation Supply",
"Total Supply", "Max Supply", "ATH", "ATL")
VALUES (1, 1, 850000000000, 35000000000, 19500000, 21000000, 21000000, 69000, 65),
(2, 2, 480000000000, 20000000000, 118000000, 120000000, NULL, 4800, 0.4),
(3, 3, 110000000000, 45000000000, 95000000, 97000000, NULL, 1.09, 0.57),
(4, 8, 20000000000, 2000000000, 34000000000, 45000000000, NULL, 3.10, 0.017),
(5, 7, 26000000000, 1800000000, 45000000000, 100000000000, NULL, 3.84, 0.0028),
(6, 4, 65000000000, 1500000000, 160000000, 170000000, NULL, 690, 0.096),
(7, 5, 59000000000, 2000000000, 440000000, 550000000, NULL, 260, 0.5),
(8, 9, 90000000000, 800000000, 1200000000, 1300000000, NULL, 55, 2.70),
(9, 10, 7000000000, 600000000, 160000000, NULL, NULL, NULL, NULL),
(10, 11, 6000000000, 500000000, 600000000, NULL, NULL, NULL, NULL);
```

```
INSERT INTO "Fiat_Currency" ("Currency ID", "Origin")
VALUES (11, 'USA'),
(12, 'European Union'),
(13, 'Japan'),
(14, 'United Kingdom'),
(15, 'Turkey'),
(16, 'Switzerland');
```

```
INSERT INTO "Market"
("Market ID", "Name", "Status", "Creation Time")
VALUES (1, 'BTC/USDT', 'OPEN', '2024-06-01'),
(2, 'ETH/USDT', 'OPEN', '2024-06-01'),
(3, 'ADA/USDT', 'OPEN', '2024-06-02'),
(4, 'XRP/USDT', 'OPEN', '2024-06-02'),
(5, 'BNB/USDT', 'OPEN', '2024-06-03'),
(6, 'SOL/USDT', 'OPEN', '2024-06-03'),
(7, 'DOT/USDT', 'OPEN', '2024-06-04'),
(8, 'BTC/EUR', 'OPEN', '2024-06-04'),
(9, 'ETH/USD', 'OPEN', '2024-06-05'),
(10, 'USD/EUR', 'OPEN', '2024-06-05');
```

```
INSERT INTO "Order_Book" ("Market ID", "Update Time")
VALUES (1, '2025-04-28 18:00'),
(2, '2025-04-28 18:00'),
(3, '2025-04-28 18:00'),
(4, '2025-04-28 18:00'),
(5, '2025-04-28 18:00'),
(6, '2025-04-28 18:00'),
(7, '2025-04-28 18:00'),
(8, '2025-04-28 18:00'),
(9, '2025-04-28 18:00'),
(10, '2025-04-28 18:00');
```

```
INSERT INTO "Trade"
("Trade ID", "Price", "Quantity", "Trade Time", "Market ID")
VALUES (1, 63000, 0.15, '2025-04-28 17:58', 1),
(2, 3200, 1.2, '2025-04-28 17:59', 2),
(3, 0.60, 1000, '2025-04-28 17:57', 3),
(4, 0.52, 1500, '2025-04-28 17:56', 4),
(5, 570, 10, '2025-04-28 17:54', 5),
(6, 150, 20, '2025-04-28 17:53', 6),
(7, 44, 50, '2025-04-28 17:52', 7),
(8, 63000, 0.12, '2025-04-28 17:51', 8),
(9, 3200, 0.8, now(), 9),
(10, 1.08, 500, '2025-04-28 17:49', 10);
```

```
INSERT INTO "Order"
("Order ID", "Type", "Status", "Quantity", "Price", "Creation Time", "Close Time",
"User ID", "Market ID", "Trade ID", "Market ID (Book)")
VALUES (1, 'BUY', 'FILLED', 0.15, 63000, '2025-04-28 17:58', '2025-04-28 17:58', 1, 1, 1, 1),
(2, 'SELL', 'FILLED', 1.2, 3200, '2025-04-28 17:59', '2025-04-28 17:59', 2, 2, 2, 2),
(3, 'BUY', 'OPEN', 500, 0.6, '2025-04-28 17:57', NULL, 3, 3, 3, 3),
(4, 'BUY', 'CANCELED', 1500, 0.52, '2025-04-28 17:56', '2025-04-28 17:56', 4, 4, 4, 4),
(5, 'SELL', 'OPEN', 10, 570, '2025-04-28 17:54', NULL, 5, 5, 5, 5),
(6, 'BUY', 'OPEN', 20, 150, '2025-04-28 17:53', NULL, 6, 6, 6, 6),
(7, 'SELL', 'FILLED', 50, 44, '2025-04-28 17:52', '2025-04-28 17:52', 7, 7, 7, 7),
(8, 'BUY', 'OPEN', 0.12, 63000, '2025-04-28 17:51', NULL, 8, 8, 8, 8),
(9, 'SELL', 'OPEN', 0.8, 3200, '2025-04-28 17:50', NULL, 9, 9, 9, 9),
(10, 'BUY', 'OPEN', 500, 1.08, '2025-04-28 17:49', NULL, 10, 10, 10, 10);
```

```
INSERT INTO "Issue"
("Issue ID", "Type", "Creation Time", "Description", "Resolve Time", "Status",
"User ID", "Agent ID")
```

```
VALUES (1, 'KYC', '2025-04-20 10:00', 'Document blurry', '2025-04-22 15:00', 'RESOLVED', 2, 1),
(2, 'Withdrawal', '2025-04-21 09:00', 'Pending too long', NULL, 'OPEN', 3, 2),
(3, 'Login', '2025-04-21 12:00', '2FA failed', '2025-04-21 13:00', 'RESOLVED', 1, 3),
(4, 'Deposit', '2025-04-22 08:00', 'Bank rejected', '2025-04-24 16:00', 'RESOLVED', 4, 4),
(5, 'Trade', '2025-04-23 11:00', 'Wrong price', NULL, 'RESOLVED', 5, 5),
(6, 'API', '2025-04-24 14:00', 'Rate limit', '2025-04-25 14:30', 'RESOLVED', 6, 6),
(7, 'KYC', '2025-04-24 15:00', 'Address proof', '2025-04-26 15:00', 'RESOLVED', 7, 7),
(8, 'Security', '2025-04-25 10:00', 'Phishing email', NULL, 'OPEN', 8, 8),
(9, 'Withdrawal', '2025-04-25 12:00', 'Tx stuck', NULL, 'OPEN', 9, 9),
(10, 'Other', '2025-04-26 09:00', 'Feedback', '2025-04-26 10:00', 'RESOLVED', 10, 10);
```

```
INSERT INTO "Transaction"
("Transaction ID", "Type", "Currency", "Amount", "Status", "Request Time", "Aprove Time",
"User ID", "Agent ID")
VALUES (1, 'DEPOSIT', 'USD', 5000, 'COMPLETED', '2025-04-20 09:00', '2025-04-20 09:05', 1, 1),
(2, 'WITHDRAW', 'USD', 0.1, 'PENDING', '2025-04-21 10:00', NULL, 2, 2),
(3, 'TRADE', 'USD', 2, 'COMPLETED', '2025-04-21 11:00', '2025-04-21 11:02', 3, 5),
(4, 'DEPOSIT', 'USD', 1000, 'COMPLETED', '2025-04-22 08:30', '2025-04-22 08:35', 4, 5),
(5, 'WITHDRAW', 'USD', 300, 'FAILED', '2025-04-22 09:00', '2025-04-22 10:00', 5, 5),
(6, 'DEPOSIT', 'USD', 2000, 'COMPLETED', '2025-04-23 14:00', '2025-04-23 14:01', 6, 6),
(7, 'TRADE', 'USD', 5, 'COMPLETED', '2025-04-23 15:00', '2025-04-23 15:02', 7, 7),
(8, 'WITHDRAW', 'USD', 10, 'PENDING', '2025-04-24 12:00', NULL, 8, 8),
(9, 'DEPOSIT', 'USD', 1500, 'COMPLETED', '2025-04-25 16:00', '2025-04-25 16:05', 9, 9),
(10, 'TRADE', 'USD', 50, 'COMPLETED', '2025-04-26 13:00', '2025-04-26 13:05', 10, 10);
```

```
INSERT INTO "Wallet_has_Currency"
("Wallet ID", "Currency ID", "Quantity")
VALUES (1, 1, 0.25),
(2, 2, 4.0),
(3, 3, 5000),
(4, 4, 2000),
(5, 5, 1500),
(6, 6, 6000),
(7, 7, 10000),
(8, 8, 25),
(9, 9, 100),
(10, 10, 300);
```

```
INSERT INTO "Currency_has_Market"
("Currency ID", "Market ID")
VALUES (1, 1),
(2, 2),
(6, 3),
(7, 4),
(8, 5),
(9, 6),
(10, 7),
(1, 8),
(2, 9),
(4, 10);
```

```
INSERT INTO "Price_History"
("Price ID", "Market ID", "High Price", "Low Price", "Open Price", "Close Price",
"Open Time", "Close Time")
VALUES (1, 1, 63500, 62000, 62500, 63000, '2025-04-28 17:00', '2025-04-28 17:59'),
(2, 2, 3300, 3100, 3150, 3200, '2025-04-28 17:00', '2025-04-28 17:59'),
(3, 3, 0.65, 0.55, 0.58, 0.60, '2025-04-28 17:00', '2025-04-28 17:59'),
(4, 4, 0.55, 0.48, 0.50, 0.52, '2025-04-28 17:00', '2025-04-28 17:59'),
(5, 5, 580, 560, 565, 570, '2025-04-28 17:00', '2025-04-28 17:59'),
(6, 6, 160, 140, 145, 150, '2025-04-28 17:00', '2025-04-28 17:59'),
(7, 7, 46, 40, 42, 44, '2025-04-28 17:00', '2025-04-28 17:59'),
(8, 8, 63500, 62000, 62500, 63000, '2025-04-28 17:00', '2025-04-28 17:59'),
(9, 9, 3300, 3100, 3150, 3200, '2025-04-28 17:00', '2025-04-28 17:59'),
(10, 10, 1.12, 1.00, 1.05, 1.08, '2025-04-28 17:00', '2025-04-28 17:59');
```

Deletion Queries

```
delete from "User" where "User ID" = 1;
delete from "Wallet" where "Status" = 'CLOSED';
delete from "Price_History" where "Price ID" = 1 and "Market ID" = 1;
delete from "Transaction" where "Status" = 'FAILED' and "Request Time" > '2023-04-19 00:00:00';
delete from "Currency_Symbol" where "Symbol" = 'ADA';
```

Update Queries

```
update "User" set "KYC Status" = 'APPROVED', "National ID" = '12345678999', "Verify Time" = now() where "User ID" = 2;
update "Wallet" set "Status" = 'ACTIVE' where "Wallet ID" = 1;
update "Order" set "Status" = 'FILLED' where "Order ID" = 3;
update "Crypto_Currency" set "Market Cap" = 5000000000,
                             "Trading Volume" = 1000000000,
                             "Circulation Supply" = 1000000000,
                             "Max Supply" = 2000000000,
                             "ATH" = 1000,
                             "ATL" = 0.1
                             where "Currency ID" = 5;
update "Price_History" set "High Price" = 2000,
                           "Low Price" = 1000,
                           "Open Price" = 1500,
                           "Close Price" = 1800
                           where "Price ID" = 2 and "Market ID" = 2;
```

Selection Queries

1. List all currencies that has the status inactive

```
select *
from "Currency"
where "Status" = 'INACTIVE';
```

```
Currency ID | Name | Symbol | Status
-----+-----+-----+-----
          7 | Ripple | XRP    | INACTIVE
(1 row)
```

2. List all orders that has the status open

```
select *
from "Order"
where "Status" = 'OPEN';
```

```
Order ID | Type | Status | Quantity | Price | Creation Time | Close Time | User ID | Market ID | Trade ID | Market ID (Book)
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
        3 | BUY | OPEN  |      500 |    0.6 | 2025-04-28 17:57:00 |           |      3 |         3 |         3 |              3
        5 | SELL | OPEN  |       10 |    570 | 2025-04-28 17:54:00 |           |      5 |         5 |         5 |              5
        6 | BUY | OPEN  |       20 |    150 | 2025-04-28 17:53:00 |           |      6 |         6 |         6 |              6
        8 | BUY | OPEN  |    0.12 | 63000 | 2025-04-28 17:51:00 |           |      8 |         8 |         8 |              8
        9 | SELL | OPEN  |     0.8 |   3200 | 2025-04-28 17:50:00 |           |      9 |         9 |         9 |              9
       10 | BUY | OPEN  |      500 |    1.08 | 2025-04-28 17:49:00 |           |     10 |        10 |        10 |             10
(6 rows)
```

3. List all user that is not verified

```
select *  
from "User"  
where "KYC Status" is NULL or "KYC Status" = 'PENDING' or "KYC Status" = 'REJECTED';
```

User ID	Email	Name	Surname	Phone	Adress	Password	Verify Time	National ID	KYC Status
2	baris_234@gmail.com	Baris	Yılmaz	+90532222222	İstanbul	r2d2a3			PENDING
3	canan_34@gmail.com	Canan	Demir	+90533333333	İzmir	Boyozbest1			PENDING
4	canberk@gmail.com	Canberk	Kaya	+90534444444	Bursa	Canberking01			
6	fransizasker2@gmail.com	Firat	Çelik	+90536666666	Konya	m4a1_s			REJECTED
7	güler_ayd@gmail.com	Güler	Aydın	+90537777777	Eskişehir	OracleBest			PENDING
8	Polat_alimdar@gmail.com	Alemdar	Polat	+90538888888	Trabzon	Cantik_63			REJECTED
10	Ali_dayi@gmail.com	Ali	Buğra	+90530000000	Antalya	351wasBetter			PENDING

(7 rows)

4. List Name, Phone and Email of users that has a total balance over 50 million

```
select u."Name", u."Phone", u."Email", w."Total Balance"  
from "User" u  
join "Wallet" w on u."User ID" = w."User ID"  
where w."Total Balance" > 50000000;
```

Name	Phone	Email	Total Balance
Firat	+9053666666666	fransizasker2@gmail.com	97557600

(1 row)

5. List all cryptocurrencies sorted by market cap

```
select cc."Currency ID", c."Name", c."Symbol", cc."Market Cap"  
from "Crypto_Currency" cc  
join "Currency" c on c."Currency ID" = cc."Currency ID"  
order by cc."Market Cap" desc;
```

Currency ID	Name	Symbol	Market Cap
1	Bitcoin	BTC	850000000000
2	Ethereum	ETH	480000000000
3	Tether USD	USDT	110000000000
6	Cardano	ADA	65000000000
7	Ripple	XRP	59000000000
5	Euro	EUR	26000000000
4	US Dollar	USD	20000000000
8	Binance Coin	BNB	9000000000
9	Solana	SOL	7000000000
10	Polkadot	DOT	6000000000

(10 rows)

6. List BTC's all market pairs that is open

```
select m."Market ID", m."Name", m."Status", m."Creation Time"
from "Currency" c
    join "Currency_has_Market" cm on cm."Curency ID" = c."Currency ID"
    join "Market" m on m."Market ID" = cm."Market ID"
where c."Symbol" = 'BTC' and m."Status" = 'OPEN';
```

Market ID	Name	Status	Creation Time
1	BTC/USDT	OPEN	2024-06-01 00:00:00
8	BTC/EUR	OPEN	2024-06-04 00:00:00

(2 rows)

7. List the five most users that made the most transaction in the last year

```
select t."User ID", u."Name", u."Surname", SUM(t."Amount") "Total Transaction Amount"
from "Transaction" t
    join "User" u on u."User ID" = t."User ID"
where t."Request Time" >= CURRENT_DATE - INTERVAL '1 year' and t."Status" = 'COMPLETED'
group by t."User ID", u."Name", u."Surname"
order by "Total Transaction Amount" desc
LIMIT 5;
```

User ID	Name	Surname	Total Transaction Amount
1	Aliye	Öztürk	5000
6	Firat	Çelik	2000
9	Ilhan	İnan	1500
4	Canberk	Kaya	1000
10	Ali	Buğra	50

(5 rows)

8. List the user 5's transaction average based on whole transaction that is succeeded

```
select u."User ID", u."Name", u."Email", avg(t."Amount") "Average Transaction Amount"
from "Transaction" t
    join "User" u on u."User ID" = t."User ID"
where u."User ID" = 9 and t."Status" = 'COMPLETED'
group by u."User ID", u."Name", u."Email";
```

User ID	Name	Email	Average Transaction Amount
9	Ilhan	ilhan21@gmail.com	1500

(1 row)

9. List the market 5's price average in a day

```
select m."Market ID", m."Name", avg((ph."Close Price" + ph."Close Price") / 2) "Average Price of Market"
from "Price_History" ph
  join "Market" m on m."Market ID" = ph."Market ID"
where ph."Market ID" = 5 and ph."Open Time" >= CURRENT_DATE - INTERVAL '1 day'
group by m."Market ID", m."Name";
```

Market ID	Name	Average Price of Market
5	BNB/USDT	570

(1 row)

10. List the market 4's trading volume today

```
select m."Market ID", m."Name", SUM(t."Quantity") "Trading Volume Today"
from "Trade" t
  join "Market" m on m."Market ID" = t."Market ID"
where t."Market ID" = 9 and t."Trade Time" >= date_trunc('day', now())
      and t."Trade Time" < date_trunc('day', now()) + INTERVAL '1 day'
group by m."Market ID", m."Name";
```

Market ID	Name	Trading Volume Today
9	ETH/USD	0.8

(1 row)

11. List all transactions approved by customer service agent 5

```
select t."Transaction ID", t."Type", t."Currency", t."Amount", t."Status", t."Request Time", t."Aprove Time",
a."Name" as "Name of the Agent"
from "Transaction" t
  join "Customer_Service_Agent" a on a."Agent ID" = t."Agent ID"
where t."Agent ID" = 5
      and t."Status" = 'COMPLETED';
```

Transaction ID	Type	Currency	Amount	Status	Request Time	Aprove Time	Name of the Agent
3	TRADE	USD	2	COMPLETED	2025-04-21 11:00:00	2025-04-21 11:02:00	Elif Er
4	DEPOSIT	USD	1000	COMPLETED	2025-04-22 08:30:00	2025-04-22 08:35:00	Elif Er

(2 rows)

12. List all issues that are resolved by customer service agent 5

```
select i."Issue ID", i."Type", i."Description", i."Creation Time", i."Resolve Time", i."Status", a."Name" as  
agent_name  
from "Issue" i  
    join "Customer_Service_Agent" a on a."Agent ID" = i."Agent ID"  
WHERE i."Agent ID" = 5  
    and i."Status" = 'RESOLVED'  
ORDER BY i."Resolve Time" desc;
```

Issue ID	Type	Description	Creation Time	Resolve Time	Status	agent_name
5	Trade	Wrong price	2025-04-23 11:00:00		RESOLVED	Elif Er

(1 row)

13. List all cryptocurrencies user 4 owns

```
select distinct c."Currency ID", c."Name", c."Symbol", wc."Quantity"  
from "User" as u  
    join "Wallet" as w on w."User ID" = u."User ID"  
    join "Wallet_has_Currency" as wc on wc."Wallet ID" = w."Wallet ID"  
    join "Currency" as c on c."Currency ID" = wc."Currency ID"  
    join "Crypto_Currency" as cc on cc."Currency ID" = c."Currency ID"  
where u."User ID" = 4;
```

Currency ID	Name	Symbol	Quantity
4	US Dollar	USD	2000

(1 row)

Discussion

We chose PostgreSQL as our database system mainly because it's open source, well documented, and widely used in both academic and professional environments, and also we were already somewhat familiar with it. PostgreSQL also performs well when dealing with complex queries, like in relational systems with multiple joins and constraints. Additionally, PostgreSQL supports full ACID compliance, so it handles transactions in a reliable way without sacrificing atomicity, consistency, isolation etc. which is very important for our project. It also has powerful features like foreign keys, views, triggers, and stored procedures, which made it a good fit for our project.

For testing purposes, we loaded a 10 sample record into each table to simulate basic operations and relationships. However, in a real-world scenario, the database would be much, much larger; potentially containing thousands or even millions of rows.

To improve performance, we added indexes to the most frequently accessed columns, especially those used in WHERE clauses or as foreign keys which will help speed up query execution by allowing the database to locate data more efficiently instead of scanning entire tables as a whole; which would result massive slowdowns in bigger databases.

Workload & Indexes

Workload Table	1	2	3	4	5	6	7	8	9	10	11	12	13
User			x	x			x	x					x
Wallet				x									x
Currency	x				x	x							x
Currency_Symbol													
Crypto_Currency					x								x
Fiat_Currency													
Market						x			x	x			
Trade										x			
Order		x											
Order_Book													
Transaction							x	x			x		
Issue												x	
Customer_Service_Agent											x	x	
Price_History									x				
Wallet_has_Currency													x
Currency_has_Market						x							

Most used table is User.

Changes and Modifications to the Database

Structurally the database is has not been changed, but we made very minor corrections such as capital letter names and whitespaces in table names since they caused problems with Flask and Python. Final version of the table creation, insertion and manipulation is as follows:

```
/* ----- */
/* 1) CREATE TABLE'LER */
/* ----- */

CREATE TABLE "user" (
    "user_id" SERIAL PRIMARY KEY,
    "email" VARCHAR(100) NOT NULL UNIQUE,
    "name" VARCHAR(60) NOT NULL,
    "surname" VARCHAR(60) NOT NULL,
    "phone" VARCHAR(20),
    "address" TEXT,
    "password" TEXT NOT NULL,
    "verify_time" TIMESTAMP,
    "national_id" VARCHAR(20),
    "kyc_status" VARCHAR(30)
);

CREATE TABLE "customer_service_agent" (
    "agent_id" SERIAL PRIMARY KEY,
    "name" VARCHAR(60) NOT NULL,
    "email" VARCHAR(100) NOT NULL UNIQUE,
    "phone" VARCHAR(20),
    "employment_time" TIMESTAMP NOT NULL
);

CREATE TABLE "wallet" (
    "wallet_id" SERIAL PRIMARY KEY,
    "status" VARCHAR(30),
    "total_balance" NUMERIC,
    "user_id" INTEGER NOT NULL REFERENCES "user"("user_id")
);

CREATE TABLE "currency_symbol" (
    "symbol" VARCHAR(10) PRIMARY KEY,
    "name" VARCHAR(40) NOT NULL
);

CREATE TABLE "currency" (
    "currency_id" SERIAL PRIMARY KEY,
    "name" VARCHAR(40) NOT NULL,
    "symbol" VARCHAR(10) NOT NULL REFERENCES "currency_symbol"("symbol"),
    "status" VARCHAR(20)
);

CREATE TABLE "crypto_currency" (
    "currency_id" INTEGER PRIMARY KEY REFERENCES "currency"("currency_id"),
    "rank" INTEGER,
    "market_cap" BIGINT,
    "trading_vol" BIGINT,
    "circulation" BIGINT,
    "total_supply" BIGINT,
    "max_supply" BIGINT,
    "ath" NUMERIC,
    "atl" NUMERIC
);

CREATE TABLE "fiat_currency" (
    "currency_id" INTEGER PRIMARY KEY REFERENCES "currency"("currency_id"),
    "origin" VARCHAR(60)
);

CREATE TABLE "market" (
    "market_id" SERIAL PRIMARY KEY,
    "name" VARCHAR(40) NOT NULL,
    "status" VARCHAR(20),
    "creation_time" TIMESTAMP NOT NULL
);

CREATE TABLE "order_book" (
    "market_id" INTEGER PRIMARY KEY REFERENCES "market"("market_id") ON DELETE CASCADE,
    "update_time" TIMESTAMP
);
```

```

CREATE TABLE "price_history" (
    "price_id"    INTEGER,
    "market_id"   INTEGER,
    "high_price"  NUMERIC,
    "low_price"   NUMERIC,
    "open_price"  NUMERIC,
    "close_price" NUMERIC,
    "open_time"   TIMESTAMP,
    "close_time"  TIMESTAMP,
    PRIMARY KEY ("price_id", "market_id"),
    FOREIGN KEY ("market_id") REFERENCES "market"("market_id") ON DELETE CASCADE
);

CREATE TABLE "trade" (
    "trade_id"    SERIAL PRIMARY KEY,
    "price"        NUMERIC,
    "quantity"     NUMERIC,
    "trade_time"   TIMESTAMP,
    "market_id"    INTEGER NOT NULL REFERENCES "market"("market_id")
);

CREATE TABLE "order" (
    "order_id"     SERIAL PRIMARY KEY,
    "type"         VARCHAR(10),
    "status"       VARCHAR(20),
    "quantity"     NUMERIC,
    "price"        NUMERIC,
    "creation_time" TIMESTAMP,
    "close_time"   TIMESTAMP,
    "user_id"      INTEGER NOT NULL REFERENCES "user"("user_id"),
    "market_id"    INTEGER NOT NULL REFERENCES "market"("market_id"),
    "trade_id"     INTEGER NOT NULL REFERENCES "trade"("trade_id"),
    "order_book_id" INTEGER NOT NULL REFERENCES "order_book"("market_id")
);

CREATE TABLE "issue" (
    "issue_id"     SERIAL PRIMARY KEY,
    "type"         VARCHAR(30),
    "creation_time" TIMESTAMP,
    "description"   TEXT,
    "resolve_time"  TIMESTAMP,
    "status"       VARCHAR(20),
    "user_id"      INTEGER NOT NULL REFERENCES "user"("user_id"),
    "agent_id"     INTEGER NOT NULL REFERENCES "customer_service_agent"("agent_id")
);

CREATE TABLE "transaction" (
    "transaction_id" SERIAL PRIMARY KEY,
    "type"           VARCHAR(20),
    "currency"       VARCHAR(10),
    "amount"        NUMERIC,
    "status"        VARCHAR(20),
    "request_time"   TIMESTAMP,
    "approve_time"   TIMESTAMP,
    "user_id"      INTEGER NOT NULL REFERENCES "user"("user_id"),
    "agent_id"     INTEGER NOT NULL REFERENCES "customer_service_agent"("agent_id")
);

CREATE TABLE "wallet_has_currency" (
    "wallet_id"    INTEGER NOT NULL REFERENCES "wallet"("wallet_id") ON DELETE CASCADE,
    "currency_id"  INTEGER NOT NULL REFERENCES "currency"("currency_id"),
    "quantity"     NUMERIC,
    PRIMARY KEY ("wallet_id", "currency_id")
);

CREATE TABLE "currency_has_market" (
    "currency_id"  INTEGER NOT NULL REFERENCES "currency"("currency_id") ON DELETE CASCADE,
    "market_id"    INTEGER NOT NULL REFERENCES "market"("market_id") ON DELETE CASCADE,
    PRIMARY KEY ("currency_id", "market_id")
);

/* ----- */
/* 2) INSERT'LER (ID sütunları dahil; isterseniz ID'leri atlayabilir) */
/* ----- */

/* -- User ----- */
INSERT INTO "user"
    ("user_id", "email", "name", "surname", "phone", "address", "password",
     "verify_time", "national_id", "kyc_status")
VALUES
    (1, 'aliye@gmail.com', 'Aliye', 'Öztürk', '+905311111111', 'Ankara', '4nk4r4_06',

```

```

        '2025-04-01 09:00','12345678901','APPROVED'),
(2,'baris_234@gmail.com','Baris','Yılmaz','+90532222222','İstanbul','r2d2a3',
NULL,NULL,'PENDING'),
(3,'canan_34@gmail.com','Canan','Demir','+90533333333','İzmir','Boyozbest1',
NULL,NULL,'PENDING'),
(4,'canberk@gmail.com','Canberk','Kaya','+90534444444','Bursa','Canberking01',
NULL,NULL,NULL),
(5,'fatih01@gmail.com','Fatih','Demirbilek','+90535555555','Adana','I_lovedb',
'2025-04-05 12:00','52345678901','APPROVED'),
(6,'fransizasker2@gmail.com','Firat','Çelik','+90536666666','Konya','m4a1_s',
NULL,NULL,'REJECTED'),
(7,'güler_ayd@gmail.com','Güler','Aydın','+90537777777','Eskişehir','OracleBest',
NULL,NULL,'PENDING'),
(8,'Polat_alimdar@gmail.com','Alemdar','Polat','+90538888888','Trabzon','Cantik_63',
NULL,NULL,'REJECTED'),
(9,'ilhan21@gmail.com','Ilhan','İnan','+90539999999','Samsun','Bandirma_vapurum',
'2025-04-09 15:00','92345678901','APPROVED'),
(10,'Ali_dayi@gmail.com','Ali','Buğra','+90530000000','Antalya','351wasBetter',
NULL,NULL,'PENDING');

/* -- Customer Service Agent ----- */
INSERT INTO "customer_service_agent"
("agent_id","name","email","phone","employment_time")
VALUES
(1,'Ahmet Akar','a.akar@cryptonit.com','+903121001001','2024-01-10'),
(2,'Büşra Bilgin','b.bilgin@cryptonit.com','+903121001002','2024-01-11'),
(3,'Cem Çetin','c.cetin@cryptonit.com','+903121001003','2024-01-12'),
(4,'Deniz Duman','d.duman@cryptonit.com','+903121001004','2024-01-13'),
(5,'Elif Er','e.er@cryptonit.com','+903121001005','2024-01-14'),
(6,'Furkan Fırat','f.firat@cryptonit.com','+903121001006','2024-01-15'),
(7,'Gökçe Güler','g.guler@cryptonit.com','+903121001007','2024-01-16'),
(8,'Hakan Hız','h.hiz@cryptonit.com','+903121001008','2024-01-17'),
(9,'İpek Ilgaz','i.ilgaz@cryptonit.com','+903121001009','2024-01-18'),
(10,'Kemal Kaya','k.kaya@cryptonit.com','+903121001010','2024-01-19');

/* -- Wallet ----- */
INSERT INTO "wallet"
("wallet_id","status","total_balance","user_id")
VALUES
(1,'ACTIVE',15000.00,1),
(2,'ACTIVE',7500.00,2),
(3,'SUSPENDED',0.00,3),
(4,'ACTIVE',12000.00,4),
(5,'ACTIVE',5500.00,5),
(6,'ACTIVE',97557600.00,6),
(7,'ACTIVE',31000.00,7),
(8,'ACTIVE',10000000.00,8),
(9,'ACTIVE',88000.00,9),
(10,'CLOSED',0.00,10);

/* -- Currency Symbol ----- */
INSERT INTO "currency_symbol" ("symbol","name") VALUES
('BTC','Bitcoin'),
('ETH','Ethereum'),
('USDT','Tether USD'),
('USD','US Dollar'),
('EUR','Euro'),
('ADA','Cardano'),
('XRP','Ripple'),
('BNB','Binance Coin'),
('SOL','Solana'),
('DOT','Polkadot'),
('JPY','Japanese Yen'),
('GBP','British Pound'),
('TRY','Turkish Lira'),
('CHF','Swiss Franc');

/* -- Currency ----- */
INSERT INTO "currency"
("currency_id","name","symbol","status")
VALUES
(1,'Bitcoin','BTC','ACTIVE'),
(2,'Ethereum','ETH','ACTIVE'),
(3,'Tether USD','USDT','ACTIVE'),
(4,'US Dollar','USD','ACTIVE'),
(5,'Euro','EUR','ACTIVE'),
(6,'Cardano','ADA','ACTIVE'),
(7,'Ripple','XRP','ACTIVE'),
(8,'Binance Coin','BNB','ACTIVE'),
(9,'Solana','SOL','ACTIVE'),
(10,'Polkadot','DOT','ACTIVE'),

```

```

(11,'USD','USD','ACTIVE'),
(12,'EUR','EUR','ACTIVE'),
(13,'JPY','JPY','ACTIVE'),
(14,'GBP','GBP','ACTIVE'),
(15,'TRY','TRY','ACTIVE'),
(16,'CHF','CHF','ACTIVE');

/* -- Crypto Currency ----- */
INSERT INTO "crypto_currency"
("currency_id","rank","market_cap","trading_vol","circulation",
"total_supply","max_supply","ath","atl")
VALUES
(1, 1, 850000000000, 35000000000, 19500000, 21000000, 21000000, 69000, 0.65),
(2, 2, 480000000000, 20000000000, 118000000, 120000000, NULL, 4800, 0.4),
(3, 3, 110000000000, 45000000000, 95000000, 97000000, NULL, 1.09, 0.57),
(4, 8, 20000000000, 2000000000, 3400000000, 4500000000, NULL, 3.10, 0.017),
(5, 7, 26000000000, 180000000, 4500000000, 10000000000, NULL, 3.84, 0.0028),
(6, 4, 65000000000, 150000000, 160000000, 170000000, NULL, 690, 0.096),
(7, 5, 59000000000, 200000000, 440000000, 550000000, NULL, 260, 0.5),
(8, 9, 9000000000, 80000000, 120000000, 130000000, NULL, 55, 2.70),
(9,10, 7000000000, 60000000, 160000000, NULL, NULL, NULL, NULL),
(10,11,6000000000, 50000000, 600000000, NULL, NULL, NULL, NULL);

/* -- Fiat Currency ----- */
INSERT INTO "fiat_currency" ("currency_id","origin") VALUES
(11,'USA'),
(12,'European Union'),
(13,'Japan'),
(14,'United Kingdom'),
(15,'Turkey'),
(16,'Switzerland');

/* -- Market ----- */
INSERT INTO "market"
("market_id","name","status","creation_time")
VALUES
(1,'BTC/USDT','OPEN','2024-06-01'),
(2,'ETH/USDT','OPEN','2024-06-01'),
(3,'ADA/USDT','OPEN','2024-06-02'),
(4,'XRP/USDT','OPEN','2024-06-02'),
(5,'BNB/USDT','OPEN','2024-06-03'),
(6,'SOL/USDT','OPEN','2024-06-03'),
(7,'DOT/USDT','OPEN','2024-06-04'),
(8,'BTC/EUR','OPEN','2024-06-04'),
(9,'ETH/USD','OPEN','2024-06-05'),
(10,'USD/EUR','OPEN','2024-06-05');

/* -- Order Book ----- */
INSERT INTO "order_book" ("market_id","update_time") VALUES
(1,'2025-04-28 18:00'),
(2,'2025-04-28 18:00'),
(3,'2025-04-28 18:00'),
(4,'2025-04-28 18:00'),
(5,'2025-04-28 18:00'),
(6,'2025-04-28 18:00'),
(7,'2025-04-28 18:00'),
(8,'2025-04-28 18:00'),
(9,'2025-04-28 18:00'),
(10,'2025-04-28 18:00');

/* -- Trade ----- */
INSERT INTO "trade"
("trade_id","price","quantity","trade_time","market_id")
VALUES
(1,63000,0.15,'2025-04-28 17:58',1),
(2, 3200,1.2 , '2025-04-28 17:59',2),
(3, 0.60,1000,'2025-04-28 17:57',3),
(4, 0.52,1500,'2025-04-28 17:56',4),
(5, 570 ,10 , '2025-04-28 17:54',5),
(6, 150 ,20 , '2025-04-28 17:53',6),
(7, 44 ,50 , '2025-04-28 17:52',7),
(8,63000,0.12 , '2025-04-28 17:51',8),
(9, 3200,0.8 ,NOW(), 9),
(10,1.08,500 , '2025-04-28 17:49',10);

/* -- Order ----- */
INSERT INTO "order"
("order_id","type","status","quantity","price","creation_time","close_time",
"user_id","market_id","trade_id","order_book_id")
VALUES
(1,'BUY','FILLED',0.15,63000,'2025-04-28 17:58','2025-04-28 17:58',1,1,1,1),

```

```

(2,'SELL','FILLED',1.2, 3200,'2025-04-28 17:59','2025-04-28 17:59',2,2,2,2),
(3,'BUY','OPEN', 500, 0.6 , '2025-04-28 17:57',NULL, 3,3,3,3),
(4,'BUY','CANCELED',1500,0.52,'2025-04-28 17:56','2025-04-28 17:56',4,4,4,4),
(5,'SELL','OPEN', 10, 570,'2025-04-28 17:54',NULL, 5,5,5,5),
(6,'BUY','OPEN', 20, 150,'2025-04-28 17:53',NULL, 6,6,6,6),
(7,'SELL','FILLED',50, 44,'2025-04-28 17:52','2025-04-28 17:52',7,7,7,7),
(8,'BUY','OPEN',0.12,63000,'2025-04-28 17:51',NULL, 8,8,8,8),
(9,'SELL','OPEN',0.8, 3200,'2025-04-28 17:50',NULL, 9,9,9,9),
(10,'BUY','OPEN',500, 1.08,'2025-04-28 17:49',NULL, 10,10,10,10);

/* -- Issue ----- */
INSERT INTO "issue"
("issue_id","type","creation_time","description","resolve_time","status",
"user_id","agent_id")
VALUES
(1,'KYC','2025-04-20 10:00','Document blurry','2025-04-22 15:00','RESOLVED',2,1),
(2,'Withdrawal','2025-04-21 09:00','Pending too long',NULL,'OPEN',3,2),
(3,'Login','2025-04-21 12:00','2FA failed','2025-04-21 13:00','RESOLVED',1,3),
(4,'Deposit','2025-04-22 08:00','Bank rejected','2025-04-24 16:00','RESOLVED',4,4),
(5,'Trade','2025-04-23 11:00','Wrong price',NULL,'RESOLVED',5,5),
(6,'API','2025-04-24 14:00','Rate limit','2025-04-25 14:30','RESOLVED',6,6),
(7,'KYC','2025-04-24 15:00','Address proof','2025-04-26 15:00','RESOLVED',7,7),
(8,'Security','2025-04-25 10:00','Phishing email',NULL,'OPEN',8,8),
(9,'Withdrawal','2025-04-25 12:00','Tx stuck',NULL,'OPEN',9,9),
(10,'Other','2025-04-26 09:00','Feedback','2025-04-26 10:00','RESOLVED',10,10);

/* -- Transaction ----- */
INSERT INTO "transaction"
("transaction_id","type","currency","amount","status","request_time","approve_time",
"user_id","agent_id")
VALUES
(1,'DEPOSIT','USD',5000 , 'COMPLETED', '2025-04-20 09:00', '2025-04-20 09:05',1,1),
(2,'WITHDRAW','USD',0.1 , 'PENDING', '2025-04-21 10:00',NULL, 2,2),
(3,'TRADE','USD',2 , 'COMPLETED', '2025-04-21 11:00', '2025-04-21 11:02', 3,5),
(4,'DEPOSIT','USD',1000 , 'COMPLETED', '2025-04-22 08:30', '2025-04-22 08:35', 4,5),
(5,'WITHDRAW','USD',300 , 'FAILED', '2025-04-22 09:00', '2025-04-22 10:00', 5,5),
(6,'DEPOSIT','USD',2000 , 'COMPLETED', '2025-04-23 14:00', '2025-04-23 14:01', 6,6),
(7,'TRADE','USD',5 , 'COMPLETED', '2025-04-23 15:00', '2025-04-23 15:02', 7,7),
(8,'WITHDRAW','USD',10 , 'PENDING', '2025-04-24 12:00',NULL, 8,8),
(9,'DEPOSIT','USD',1500 , 'COMPLETED', '2025-04-25 16:00', '2025-04-25 16:05', 9,9),
(10,'TRADE','USD',50 , 'COMPLETED', '2025-04-26 13:00', '2025-04-26 13:05', 10,10);

/* -- Wallet_has_Currency ----- */
INSERT INTO "wallet_has_currency"
("wallet_id","currency_id","quantity")
VALUES
(1,1,0.25),
(2,2,4.0),
(3,3,5000),
(4,4,2000),
(5,5,1500),
(6,6,6000),
(7,7,10000),
(8,8,25),
(9,9,100),
(10,10,300);

/* -- Currency_has_Market ----- */
INSERT INTO "currency_has_market"
("currency_id","market_id")
VALUES
(1,1),
(2,2),
(6,3),
(7,4),
(8,5),
(9,6),
(10,7),
(1,8),
(2,9),
(4,10);

/* -- Price History ----- */
INSERT INTO "price_history"
("price_id","market_id","high_price","low_price","open_price","close_price",
"open_time","close_time")
VALUES
(1,1,63500,62000,62500,63000,'2025-04-28 17:00','2025-04-28 17:59'),
(2,2, 3300, 3100, 3150, 3200,'2025-04-28 17:00','2025-04-28 17:59'),
(3,3, 0.65, 0.55, 0.58, 0.60,'2025-04-28 17:00','2025-04-28 17:59'),
(4,4,0.55, 0.48, 0.50, 0.52,'2025-04-28 17:00','2025-04-28 17:59'),

```

```

(5,5, 580, 560, 565, 570 , '2025-04-28 17:00', '2025-04-28 17:59'),
(6,6, 160, 140, 145, 150 , '2025-04-28 17:00', '2025-04-28 17:59'),
(7,7, 46, 40, 42, 44 , '2025-04-28 17:00', '2025-04-28 17:59'),
(8,8,63500,62000,62500,63000, '2025-04-28 17:00', '2025-04-28 17:59'),
(9,9, 3300, 3100, 3150, 3200, '2025-04-28 17:00', '2025-04-28 17:59'),
(10,10,1.12, 1.00, 1.05, 1.08, '2025-04-28 17:00', '2025-04-28 17:59');

/* ----- */
/* 3) DELETE SORGULARI */
/* ----- */

DELETE FROM "user" WHERE "user_id" = 1;
DELETE FROM "wallet" WHERE "status" = 'CLOSED';
DELETE FROM "price_history" WHERE "price_id" = 1 AND "market_id" = 1;
DELETE FROM "transaction" WHERE "status" = 'FAILED'
AND "request_time" > '2023-04-19 00:00:00';
DELETE FROM "currency_symbol" WHERE "symbol" = 'ADA';

/* ----- */
/* 4) UPDATE SORGULARI */
/* ----- */

UPDATE "user"
SET "kyc_status" = 'APPROVED',
    "national_id" = '12345678999',
    "verify_time" = NOW()
WHERE "user_id" = 2;

UPDATE "wallet"
SET "status" = 'ACTIVE'
WHERE "wallet_id" = 1;

UPDATE "order"
SET "status" = 'FILLED'
WHERE "order_id" = 3;

UPDATE "crypto_currency"
SET "market_cap" = 5000000000,
    "trading_vol" = 1000000000,
    "circulation" = 1000000000,
    "max_supply" = 2000000000,
    "ath" = 1000,
    "atl" = 0.1
WHERE "currency_id" = 5;

UPDATE "price_history"
SET "high_price" = 2000,
    "low_price" = 1000,
    "open_price" = 1500,
    "close_price" = 1800
WHERE "price_id" = 2 AND "market_id" = 2;

/* ----- */
/* 5) SELECT SORGULARI */
/* ----- */

/* 1. Durumu 'INACTIVE' olan tüm paralar */
SELECT * FROM "currency" WHERE "status" = 'INACTIVE';

/* 2. Durumu 'OPEN' olan tüm emirler */
SELECT * FROM "order" WHERE "status" = 'OPEN';

/* 3. Henüz doğrulanmamış kullanıcılar */
SELECT *
FROM "user"
WHERE "kyc_status" IS NULL
OR "kyc_status" IN ('PENDING', 'REJECTED');

/* 4. Bakiye > 50 000 000 olan kullanıcıların adı-tel-e-posta */
SELECT u."name", u."phone", u."email", w."total_balance"
FROM "user" u
JOIN "wallet" w ON w."user_id" = u."user_id"
WHERE w."total_balance" > 50000000;

/* 5. Piyasa değerine göre kripto paralar */
SELECT cc."currency_id", c."name", c."symbol", cc."market_cap"
FROM "crypto_currency" cc
JOIN "currency" c ON c."currency_id" = cc."currency_id"
ORDER BY cc."market_cap" DESC;

/* 6. BTC'nin açık market çiftleri */

```

```

SELECT m."market_id", m."name", m."status", m."creation_time"
FROM "currency" c
JOIN "currency_has_market" cm ON cm."currency_id" = c."currency_id"
JOIN "market" m ON m."market_id" = cm."market_id"
WHERE c."symbol" = 'BTC' AND m."status" = 'OPEN';

/* 7. Son 1 yılda en yüksek toplam işlem tutarına sahip 5 kullanıcı */
SELECT t."user_id", u."name", u."surname",
SUM(t."amount") AS "total_transaction_amount"
FROM "transaction" t
JOIN "user" u ON u."user_id" = t."user_id"
WHERE t."request_time" >= CURRENT_DATE - INTERVAL '1 year'
AND t."status" = 'COMPLETED'
GROUP BY t."user_id", u."name", u."surname"
ORDER BY "total_transaction_amount" DESC
LIMIT 5;

/* 8. Kullanıcı 9'un (örnekte) başarılı işlemlerinin ortalaması */
SELECT u."user_id", u."name", u."email",
AVG(t."amount") AS "average_transaction_amount"
FROM "transaction" t
JOIN "user" u ON u."user_id" = t."user_id"
WHERE u."user_id" = 9
AND t."status" = 'COMPLETED'
GROUP BY u."user_id", u."name", u."email";

/* 9. Market 5'in son 24 saatlik ortalama kapanış fiyatı */
SELECT m."market_id", m."name",
AVG(ph."close_price") AS "average_price_of_market"
FROM "price_history" ph
JOIN "market" m ON m."market_id" = ph."market_id"
WHERE ph."market_id" = 5
AND ph."open_time" >= CURRENT_DATE - INTERVAL '1 day'
GROUP BY m."market_id", m."name";

/* 10. Market 9'un (soruda 4 denmişti) bugünkü işlem hacmi */
SELECT m."market_id", m."name",
SUM(t."quantity") AS "trading_volume_today"
FROM "trade" t
JOIN "market" m ON m."market_id" = t."market_id"
WHERE t."market_id" = 9
AND t."trade_time" >= DATE_TRUNC('day', NOW())
AND t."trade_time" < DATE_TRUNC('day', NOW()) + INTERVAL '1 day'
GROUP BY m."market_id", m."name";

/* 11. Müşteri temsilcisi 5'in onayladığı işlemler */
SELECT t."transaction_id", t."type", t."currency", t."amount",
t."status", t."request_time", t."approve_time",
a."name" AS agent_name
FROM "transaction" t
JOIN "customer_service_agent" a ON a."agent_id" = t."agent_id"
WHERE t."agent_id" = 5
AND t."status" = 'COMPLETED';

/* 12. Müşteri temsilcisi 5'in çözdüğü talepler */
SELECT i."issue_id", i."type", i."description",
i."creation_time", i."resolve_time", i."status",
a."name" AS agent_name
FROM "issue" i
JOIN "customer_service_agent" a ON a."agent_id" = i."agent_id"
WHERE i."agent_id" = 5
AND i."status" = 'RESOLVED'
ORDER BY i."resolve_time" DESC;

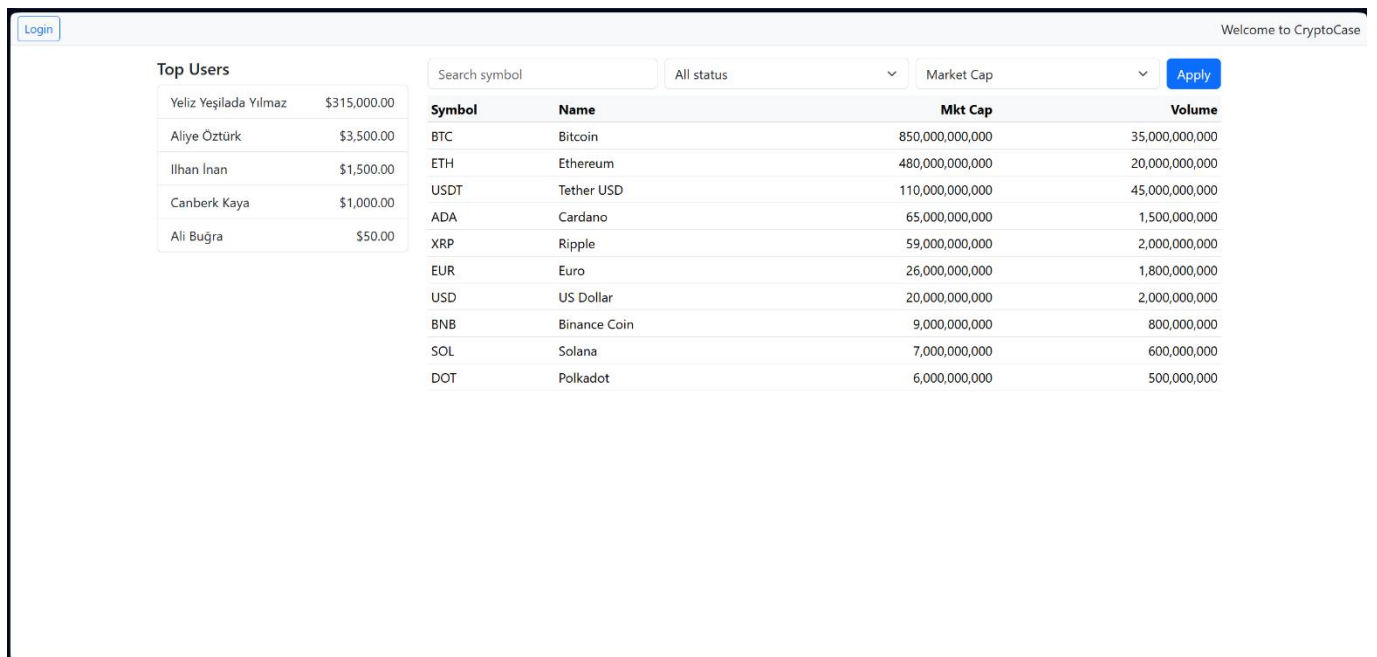
/* 13. Kullanıcı 4'ün sahip olduğu kripto paralar */
SELECT DISTINCT c."currency_id", c."name", c."symbol", wc."quantity"
FROM "user" u
JOIN "wallet" w ON w."user_id" = u."user_id"
JOIN "wallet_has_currency" wc ON wc."wallet_id" = w."wallet_id"
JOIN "currency" c ON c."currency_id" = wc."currency_id"
JOIN "crypto_currency" cc ON cc."currency_id" = c."currency_id"
WHERE u."user_id" = 4;

```


Implementation Details

- For our database, we kept PostgreSQL as our preferred choice of database management system.
- We used Python3 for main functionalities of our application and plain HTML for user interface.
- For authentication and web framework we used Flask. Flask is a “micro-framework” which is more suitable for our needs compared to bigger and more complex frameworks like Django, but it gives us more control and flexibility over the application structure. Also Flask’s easy to use and implement routing system makes things much easier when mapping URLs to Python functions.

Application



The screenshot shows the main interface of the CryptoCase application. At the top left is a 'Login' button. At the top right is a 'Welcome to CryptoCase' message. Below the login button is a 'Top Users' table. Below the search bar and filters is a table of trading symbols, names, market caps, and volumes.

Symbol	Name	Mkt Cap	Volume
BTC	Bitcoin	850,000,000,000	35,000,000,000
ETH	Ethereum	480,000,000,000	20,000,000,000
USDT	Tether USD	110,000,000,000	45,000,000,000
ADA	Cardano	65,000,000,000	1,500,000,000
XRP	Ripple	59,000,000,000	2,000,000,000
EUR	Euro	26,000,000,000	1,800,000,000
USD	US Dollar	20,000,000,000	2,000,000,000
BNB	Binance Coin	9,000,000,000	800,000,000
SOL	Solana	7,000,000,000	600,000,000
DOT	Polkadot	6,000,000,000	500,000,000

This is the main screen of our application when it’s first launched. We can directly see the options for logging in, available symbols to trade, their Market Cap and Trading volume and an additional “Top Users” section where users with highest trading volume are ordered by their average trading volume size.

Trading volume accounts for trades made with different currencies as well; for example 1.0 BTC (worth 100.000\$) trade made by user Ali Buğra will be accounted for and added to the total in Top Users chart in USD form.

Login

Email

Password

Login

No account? [Register](#)

When a user clicks Login button in the main menu this screen will be presented. Users can log in if they have an account or they can register with the button below. Registering will add that user to the database.

Create account

Email

Name

Surname

Password

Register

Already have an account? [Log in](#)

If the user click register page this menu will show up. Registered Name and Surname will be visible on main screen when that user logs in. If they landed on this page by mistake, they can return back to log in menu with the “Already have an account? [Log in](#)” prompt.

[Logout](#)

[Create Issue](#)

Ali Buğra Şahin

[Wallet](#)

Top Users

Yeliz Yeşilada Yılmaz	\$315,000.00
Aliye Öztürk	\$3,500.00
Ilhan Inan	\$1,500.00
Canberk Kaya	\$1,000.00
Ali Buğra	\$50.00

All status

Market Cap

Apply

Symbol	Name	Mkt Cap	Volume
BTC	Bitcoin	850,000,000,000	35,000,000,000
ETH	Ethereum	480,000,000,000	20,000,000,000
USDT	Tether USD	110,000,000,000	45,000,000,000
ADA	Cardano	65,000,000,000	1,500,000,000
XRP	Ripple	59,000,000,000	2,000,000,000
EUR	Euro	26,000,000,000	1,800,000,000
USD	US Dollar	20,000,000,000	2,000,000,000
BNB	Binance Coin	9,000,000,000	800,000,000
SOL	Solana	7,000,000,000	600,000,000
DOT	Polkadot	6,000,000,000	500,000,000

Markets

Home page after logging it with registered user alibugrasahin06@hotmail.com. You can see the name visible at top right corner with “Create Issue” and “Wallet” options. Also new “Markets” tab will be available below the symbols, which will show the open markets.

[Logout](#)

[Create Issue](#)

Ali Buğra Şahin

[Wallet](#)

Top Users

Yeliz Yeşilada Yılmaz	\$315,000.00
Aliye Öztürk	\$3,500.00
Ilhan Inan	\$1,500.00
Canberk Kaya	\$1,000.00
Ali Buğra	\$50.00

All status

Market Cap

Apply

Symbol	Name	Market Cap	Trading Vol	Volume
BTC	Bitcoin	850,000,000,000	35,000,000,000	35,000,000,000
ETH	Ethereum	480,000,000,000	20,000,000,000	20,000,000,000
USDT	Tether USD	110,000,000,000	45,000,000,000	45,000,000,000
ADA	Cardano	65,000,000,000	1,500,000,000	1,500,000,000
XRP	Ripple	59,000,000,000	2,000,000,000	2,000,000,000
EUR	Euro	26,000,000,000	1,800,000,000	1,800,000,000
USD	US Dollar	20,000,000,000	2,000,000,000	2,000,000,000
BNB	Binance Coin	9,000,000,000	800,000,000	800,000,000
SOL	Solana	7,000,000,000	600,000,000	600,000,000
DOT	Polkadot	6,000,000,000	500,000,000	500,000,000

Markets

Users can change the sorting order of symbols either by Market Cap or Trading Volume.

Logout

Create Issue

Ali Buğra Şahin

Wallet

Logout

Top Users

Yeliz Yeşilada Yılmaz	\$315,000.00
Aliye Öztürk	\$3,500.00
Ilhan İnan	\$1,500.00
Canberk Kaya	\$1,000.00
Ali Buğra	\$50.00

Search symbol

All status

All status

Active

Inactive

Market Cap

Apply

Symbol	Name	Mkt Cap	Volume
BTC	Bitcoin	50,000,000,000	35,000,000,000
ETH	Ethereum	480,000,000,000	20,000,000,000
USDT	Tether USD	110,000,000,000	45,000,000,000
ADA	Cardano	65,000,000,000	1,500,000,000
XRP	Ripple	59,000,000,000	2,000,000,000
EUR	Euro	26,000,000,000	1,800,000,000
USD	US Dollar	20,000,000,000	2,000,000,000
BNB	Binance Coin	9,000,000,000	800,000,000
SOL	Solana	7,000,000,000	600,000,000
DOT	Polkadot	6,000,000,000	500,000,000

Markets

Users can filter down symbols by their status: Active or Inactive. Inactive symbols can not be traded.

Logout

Create Issue

Ali Buğra Şahin

Wallet

Logout

Top Users

Yeliz Yeşilada Yılmaz	\$315,000.00
Aliye Öztürk	\$3,500.00
Ilhan İnan	\$1,500.00
Canberk Kaya	\$1,000.00
Ali Buğra	\$50.00

Search symbol

Active

Trading Vol

Apply

Symbol	Name	Mkt Cap	Volume
USDT	Tether USD	110,000,000,000	45,000,000,000
BTC	Bitcoin	850,000,000,000	35,000,000,000
ETH	Ethereum	480,000,000,000	20,000,000,000
USD	US Dollar	20,000,000,000	2,000,000,000
XRP	Ripple	59,000,000,000	2,000,000,000
EUR	Euro	26,000,000,000	1,800,000,000
DOT	Polkadot	6,000,000,000	500,000,000

Markets

Main menu after changing sorting type from Market Cap to Trading Volume (ascending) and filtering symbols by active only.

Create Issue

Type

Choose... ▾

Description

Submit Issue

If logged in user clicks “Create Issue” button in the homepage this page will be presented to them.

Create Issue

Type

Choose... ▾

Choose...

KYC

Withdrawal

Login

Deposit

Trade

API

Security

Other

Users can pick a issue type from the drop-down menu and write their problem in Description. Any submitted issues will be sent to the database for Customer Service Agent to check and handle.

Create Issue

Type

Login

Description

Unable to login. System don't accept the given email.

Submit Issue

Let's try to submit a login issue.

consolewallet_has_currencytransactionissue

WHEREORDER BY

issue_id	type	creation_time	description	resolve_time	status	user_id	agent_id
1	KYC	2025-04-20 10:00:00.000000	Document blurry	2025-04-22 15:00:00.000000	RESOLVED	2	1
2	Withdrawal	2025-04-21 09:00:00.000000	Pending too long	<null>	OPEN	3	2
3	Login	2025-04-21 12:00:00.000000	2FA failed	2025-04-21 13:00:00.000000	RESOLVED	1	3
4	Deposit	2025-04-22 08:00:00.000000	Bank rejected	2025-04-24 16:00:00.000000	RESOLVED	4	4
5	Trade	2025-04-23 11:00:00.000000	Wrong price	<null>	RESOLVED	5	5
6	API	2025-04-24 14:00:00.000000	Rate limit	2025-04-25 14:30:00.000000	RESOLVED	6	6
7	KYC	2025-04-24 15:00:00.000000	Address proof	2025-04-26 15:00:00.000000	RESOLVED	7	7
8	Security	2025-04-25 10:00:00.000000	Phishing email	<null>	OPEN	8	8
9	Withdrawal	2025-04-25 12:00:00.000000	Tx stuck	<null>	OPEN	9	9
10	Other	2025-04-26 09:00:00.000000	Feedback	2025-04-26 10:00:00.000000	RESOLVED	10	10
11	KYC	2025-05-22 23:11:57.609830	asegfasdgd	<null>	OPEN	35	1
12	KYC	2025-05-22 23:13:37.559622	AAAAAAAAAAAAAAAAAAAA	<null>	OPEN	35	1
13	Login	2025-05-22 23:14:17.978474	beni iceri alin	<null>	OPEN	35	1
14	Login	2025-05-23 16:42:33.803992	oajbfoadbnf	<null>	OPEN	35	1

14 rows

Database

postgres@localhost1 of 2

postgres1 of 3

public

tables 17

alembic_version

crypto_currency

currency

currency_has_market

currency_symbol

customer_service_agent

fiat_currency

issue

market

order

order_book

price_history

trade

transaction

user

wallet

wallet_has_currency

sequences 0

Database Objects

Server Objects

This is the issue table before the issue is submitted.

issue_id	type	creation_time	description	resolve_time	status	user_id	agent_id
1	KYC	2025-04-20 10:00:00.000000	Document blurry	2025-04-22 15:00:00.000000	RESOLVED	2	1
2	Withdrawal	2025-04-21 09:00:00.000000	Pending too long	<null>	OPEN	3	2
3	Login	2025-04-21 12:00:00.000000	2FA failed	2025-04-21 13:00:00.000000	RESOLVED	1	3
4	Deposit	2025-04-22 08:00:00.000000	Bank rejected	2025-04-24 16:00:00.000000	RESOLVED	4	4
5	Trade	2025-04-23 11:00:00.000000	Wrong price	<null>	RESOLVED	5	5
6	API	2025-04-24 14:00:00.000000	Rate limit	2025-04-25 14:30:00.000000	RESOLVED	6	6
7	KYC	2025-04-24 15:00:00.000000	Address proof	2025-04-26 15:00:00.000000	RESOLVED	7	7
8	Security	2025-04-25 10:00:00.000000	Phishing email	<null>	OPEN	8	8
9	Withdrawal	2025-04-25 12:00:00.000000	Tx stuck	<null>	OPEN	9	9
10	Other	2025-04-26 09:00:00.000000	Feedback	2025-04-26 10:00:00.000000	RESOLVED	10	10
11	KYC	2025-05-22 23:11:57.609830	asegfasdggf	<null>	OPEN	35	1
12	KYC	2025-05-22 23:13:37.559622	AAAAAAAAAAAAAAAAAAAA	<null>	OPEN	35	1
13	Login	2025-05-22 23:14:17.978474	beni içeri a1n	<null>	OPEN	35	1
14	Login	2025-05-23 16:42:33.803992	oajbfoadbnf	<null>	OPEN	35	1
15	Login	2025-05-25 12:00:08.657856	Unable to login. Syste	<null>	OPEN	35	1

After the issue submission, it will be added to the issue table. Here, you can see the issue type, the description of the issue and the current status of the issue (OPEN or RESOLVED).

Logout
Create Issue
Ali Buğra Şahin

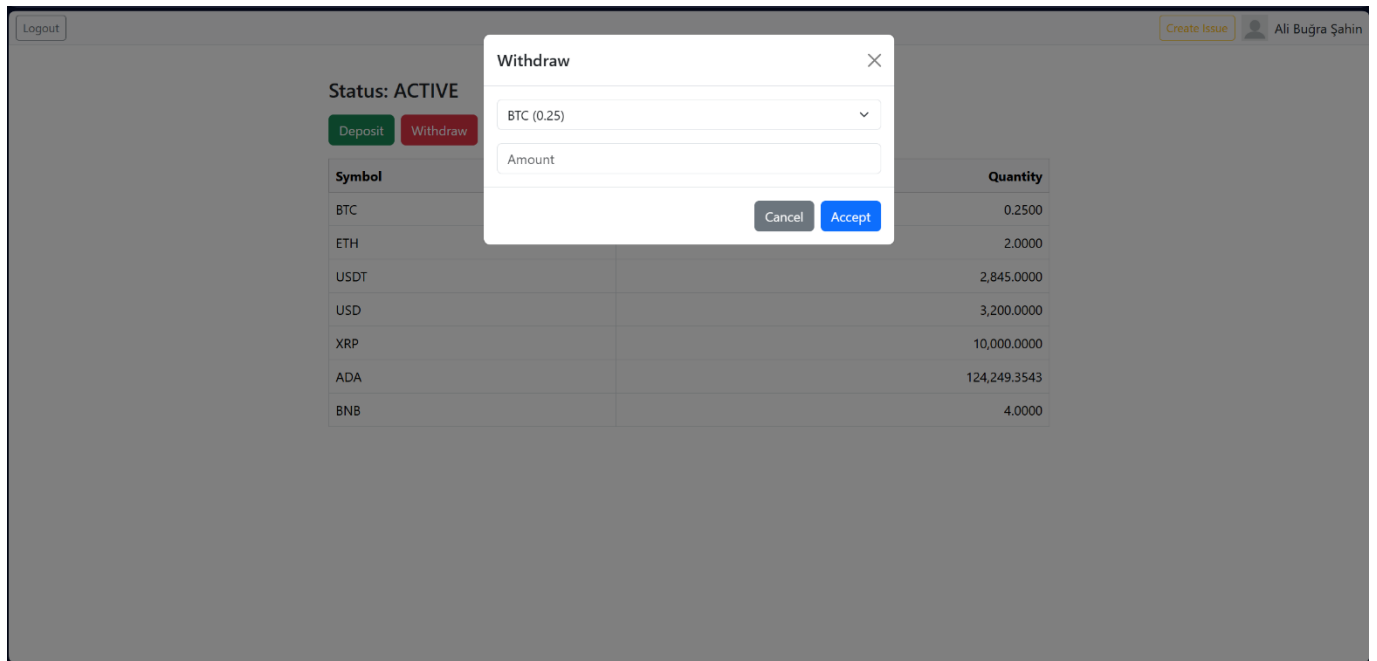
Status: ACTIVE

Deposit
Withdraw

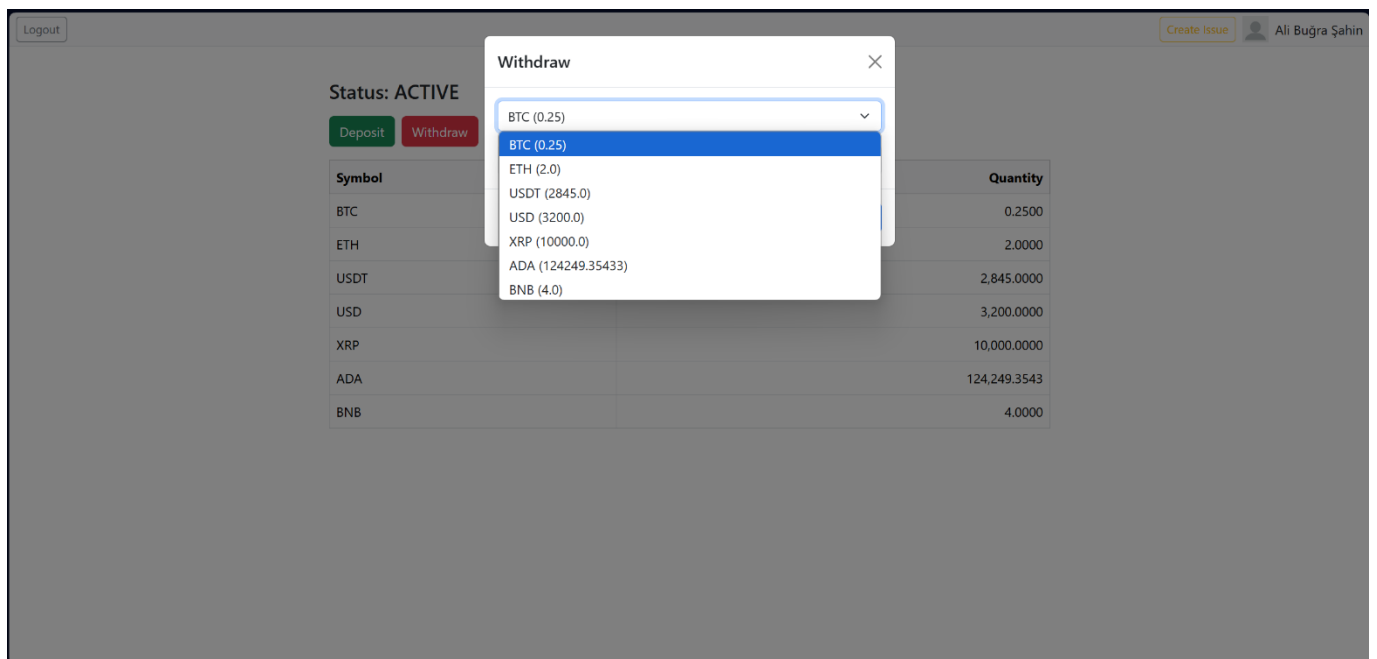
Symbol	Quantity
BTC	0.2500
ETH	2.0000
USDT	2,845.0000
USD	3,200.0000
XRP	10,000.0000
ADA	124,249.3543
BNB	4.0000

If registered user clicks wallet, this page will be presented. For example user Ali Buğra Şahin has an active wallet with these symbols. They can select the option to deposit new symbol or more of a owned symbol, or withdraw any symbol they own.

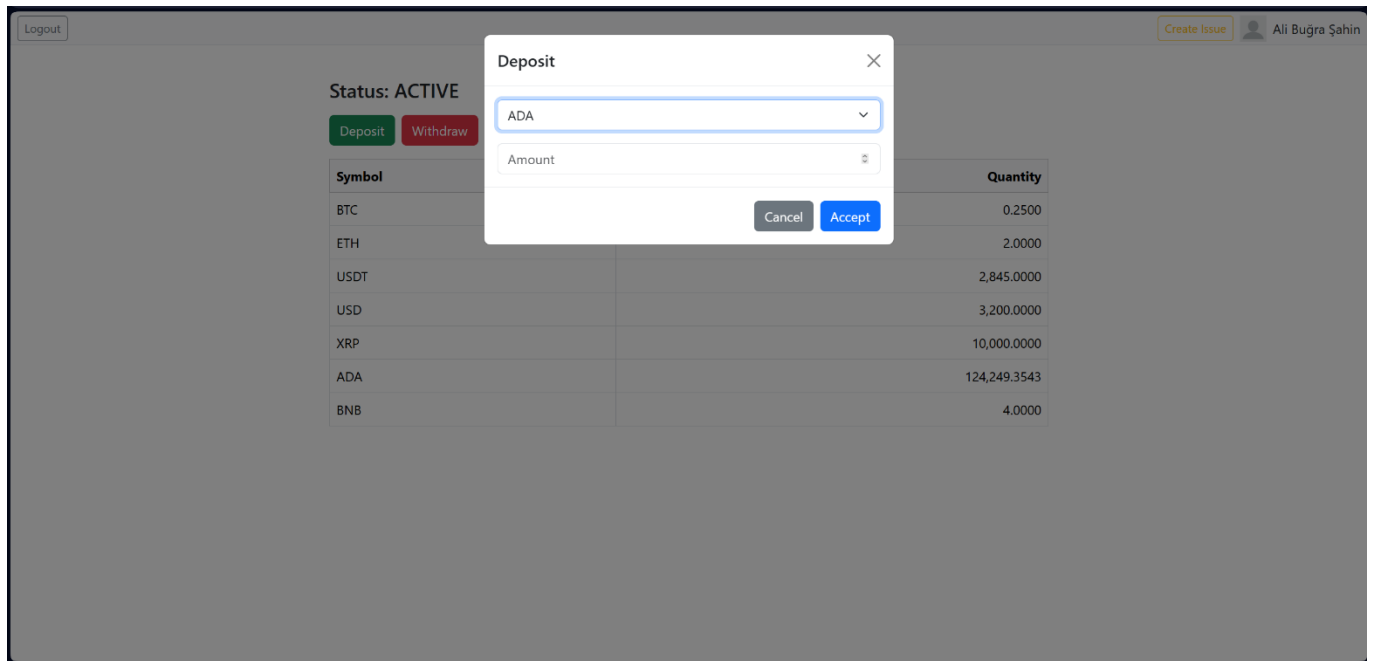
Users with inactive wallets can't deposit or withdraw from their account.



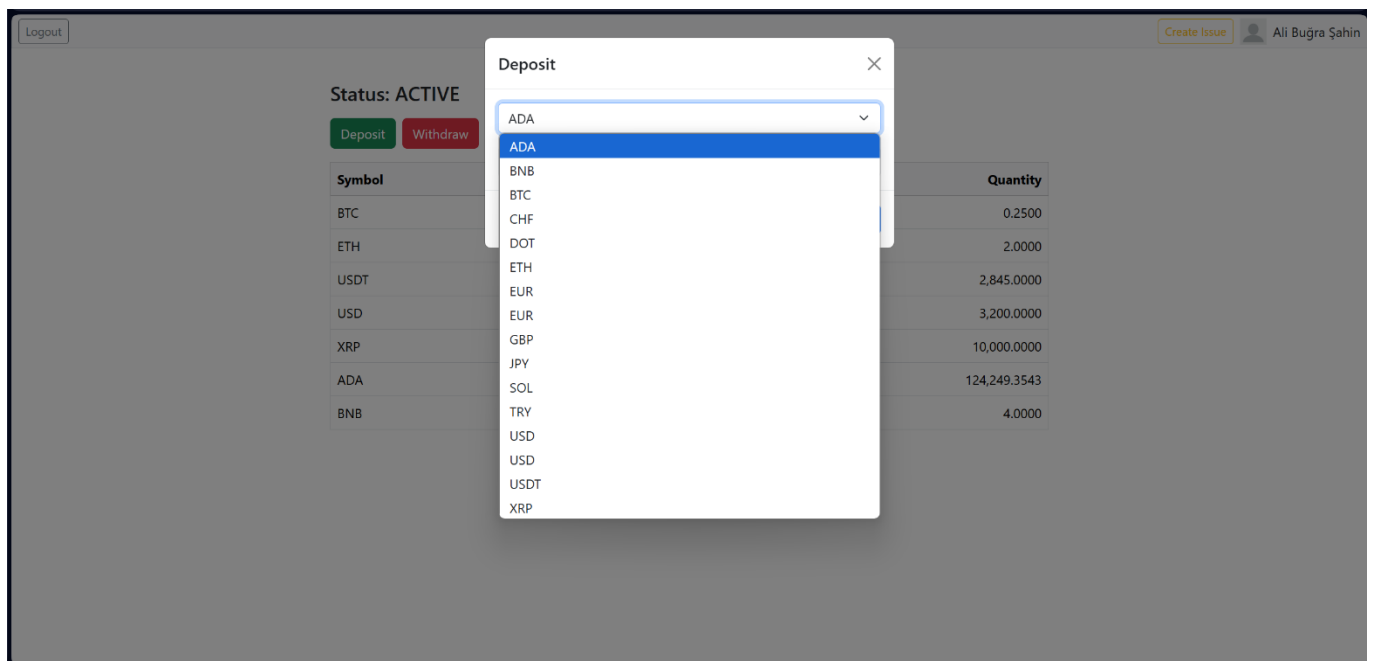
Clicking withdraw will present user with this menu.



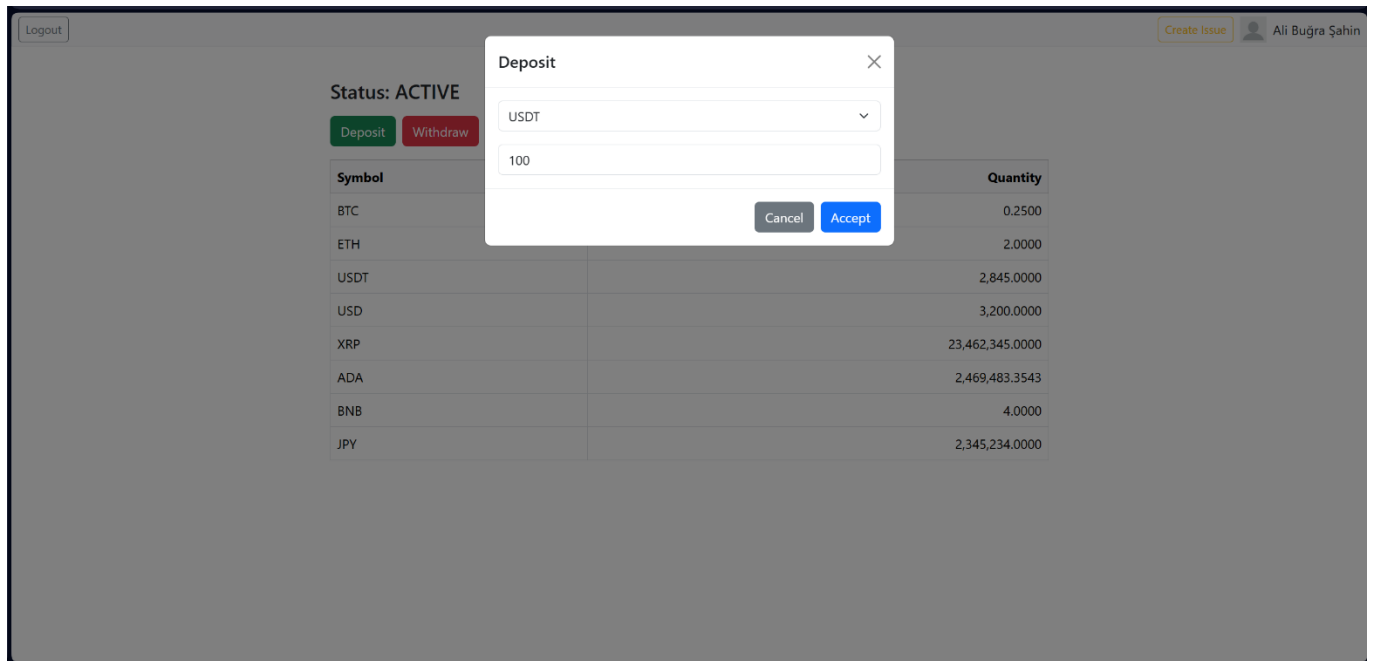
User can specify which symbol to withdraw and the amount of it from this menu. After that, they can either accept it which will be sent to database for Customer Service Agent or cancel it. Customer Service Agent will decide on proceeding or declining the request.



Clicking deposit button will present user with this menu.



Users can select which symbol to deposit and the amount they want to deposit. If enough cash (USD) is present in the account the request will be sent to the Customer Service Agent, which they will decide on accepting or denying the request.



Let's try to deposit 100 USDT to our account. User will simply type in the desired amount and click accept.

The screenshot shows a PostgreSQL database console with a table of transactions. The table has columns: transaction_id, type, currency, amount, status, request_time, approve_time, user_id, and agent_id. The table contains 23 rows of transaction data. The last row (transaction_id 23) shows a deposit of 100 EUR, which is currently in a 'PENDING' status.

transaction_id	type	currency	amount	status	request_time	approve_time	user_id	agent_id
1	DEPOSIT	USD	1000	COMPLETED	2025-04-20 09:00:00.000000	2025-04-20 09:00:00.000000	4	5
2	WITHDRAW	USD	300	FAILED	2025-04-22 09:00:00.000000	2025-04-22 10:00:00.000000	5	5
3	WITHDRAW	USD	10	PENDING	2025-04-24 12:00:00.000000	<null>	8	8
4	DEPOSIT	USD	1500	COMPLETED	2025-04-25 16:00:00.000000	2025-04-25 16:05:00.000000	9	9
5	DEPOSIT	USD	2000	COMPLETED	2025-04-23 14:00:00.000000	2025-04-23 14:01:00.000000	1	6
6	DEPOSIT	USD	50	COMPLETED	2025-04-26 13:00:00.000000	2025-04-26 13:05:00.000000	10	10
7	DEPOSIT	BTC	5	COMPLETED	2025-04-23 15:00:00.000000	2025-04-23 15:02:00.000000	7	7
8	DEPOSIT	BTC	444	COMPLETED	2025-05-24 20:38:42.357740	2025-04-23 15:02:00.000000	5	1
9	DEPOSIT	ADA	2345234	COMPLETED	2025-05-24 20:34:37.118833	2025-04-23 15:02:00.000000	35	1
10	WITHDRAWAL	USDT	2845	COMPLETED	2025-05-24 20:33:20.104738	2025-04-21 11:02:00.000000	35	1
11	DEPOSIT	ADA	1	COMPLETED	2025-05-23 01:37:58.942668	2025-04-21 11:02:00.000000	35	1
12	WITHDRAWAL	ETH	1	COMPLETED	2025-05-23 01:39:34.828512	2025-04-26 13:05:00.000000	35	1
13	DEPOSIT	ADA	1.0001	COMPLETED	2025-05-23 01:38:16.191129	2025-04-26 13:05:00.000000	35	1
14	DEPOSIT	ADA	124235.23423	COMPLETED	2025-05-23 01:38:49.449554	2025-04-23 15:02:00.000000	35	1
15	DEPOSIT	ADA	12.12	COMPLETED	2025-05-23 01:38:58.243464	2025-04-21 11:02:00.000000	35	1
16	DEPOSIT	XRP	23452345	COMPLETED	2025-05-24 20:34:42.527836	2025-04-21 11:02:00.000000	35	1
17	DEPOSIT	USDT	125	COMPLETED	2025-05-23 16:41:37.616936	2025-04-23 15:02:00.000000	35	1
18	DEPOSIT	USDT	2845	COMPLETED	2025-05-24 20:34:24.880561	2025-04-26 13:05:00.000000	35	1
19	DEPOSIT	JPY	2345234	COMPLETED	2025-05-24 20:34:49.262165	2025-04-26 13:05:00.000000	35	1
20	DEPOSIT	EUR	500000	COMPLETED	2025-05-24 20:38:33.882998	2025-04-23 15:02:00.000000	5	1
21	DEPOSIT	EUR	100	PENDING	2025-05-24 20:38:33.882998	<null>	5	1

This is the database for transactions before the accept button is pressed.

transaction_id	type	currency	amount	status	request_time	approve_time	user_id	agent_id
2	DEPOSIT	USD	1000	COMPLETED	2025-04-22 08:30:00.000000	2025-04-22 08:35:00.000000	4	5
3	WITHDRAW	USD	300	FAILED	2025-04-22 09:00:00.000000	2025-04-22 10:00:00.000000	5	5
4	WITHDRAW	USD	10	PENDING	2025-04-24 12:00:00.000000	<null>	8	8
5	DEPOSIT	USD	1500	COMPLETED	2025-04-25 16:00:00.000000	2025-04-25 16:05:00.000000	9	9
6	DEPOSIT	USD	2000	COMPLETED	2025-04-23 14:00:00.000000	2025-04-23 14:01:00.000000	1	6
7	DEPOSIT	USD	0.1	PENDING	2025-04-21 10:00:00.000000	<null>	2	2
8	DEPOSIT	USD	2	COMPLETED	2025-04-21 11:00:00.000000	2025-04-21 11:02:00.000000	3	5
9	DEPOSIT	USD	50	COMPLETED	2025-04-26 13:00:00.000000	2025-04-26 13:05:00.000000	10	10
10	DEPOSIT	BTC	5	COMPLETED	2025-04-23 15:00:00.000000	2025-04-23 15:02:00.000000	7	7
11	DEPOSIT	BTC	444	COMPLETED	2025-05-24 20:38:42.357740	2025-04-23 15:02:00.000000	5	1
12	DEPOSIT	ADA	2345234	COMPLETED	2025-05-24 20:34:37.118833	2025-04-23 15:02:00.000000	35	1
13	WITHDRAWAL	USDT	2845	COMPLETED	2025-05-24 20:33:20.104738	2025-04-21 11:02:00.000000	35	1
14	DEPOSIT	ADA	1	COMPLETED	2025-05-23 01:37:58.942668	2025-04-21 11:02:00.000000	35	1
15	WITHDRAWAL	ETH	1	COMPLETED	2025-05-23 01:39:34.828512	2025-04-26 13:05:00.000000	35	1
16	DEPOSIT	ADA	1.0001	COMPLETED	2025-05-23 01:38:16.191129	2025-04-26 13:05:00.000000	35	1
17	DEPOSIT	ADA	124235.23423	COMPLETED	2025-05-23 01:38:49.449559	2025-04-23 15:02:00.000000	35	1
18	DEPOSIT	ADA	12.12	COMPLETED	2025-05-23 01:38:58.243464	2025-04-21 11:02:00.000000	35	1
19	DEPOSIT	XRP	23452345	COMPLETED	2025-05-24 20:34:42.527836	2025-04-21 11:02:00.000000	35	1
20	DEPOSIT	USDT	125	COMPLETED	2025-05-23 16:41:37.616936	2025-04-23 15:02:00.000000	35	1
21	DEPOSIT	USDT	2845	COMPLETED	2025-05-24 20:34:24.880561	2025-04-26 13:05:00.000000	35	1
22	DEPOSIT	JPY	2345234	COMPLETED	2025-05-24 20:34:49.262165	2025-04-26 13:05:00.000000	35	1
23	DEPOSIT	EUR	500000	COMPLETED	2025-05-24 20:38:33.882998	2025-04-23 15:02:00.000000	5	1
24	DEPOSIT	USDT	100	PENDING	2025-05-25 12:03:04.497923	<null>	35	1

After user clicks accept, the request will be sent to the database as shown here. It stays as pending until Customer Service Agent approves or cancels the request.

Name	Price	
BNB/USDT	570.0	Details
BTC/EUR	63000.0	Details
BTC/USDT	63000.0	Details
DOT/USDT	44.0	Details
ETH/USD	3200.0	Details
ETH/USDT	3200.0	Details
SOL/USDT	150.0	Details
USD/EUR	1.08	Details
XRP/USDT	0.52	Details

If registered user clicks “Markets” button in the homepage they’ll be able to see the open markets and each of symbols’ conversion prices to each other. User can click details button to show more options for each symbol.

[Logout](#)[Create Issue](#)

Ali Buğra Şahin

BNB/USDT

Market Price: 570.0
24-h Trading Volume 14.0

BNB:

USDT:

Limit:

BuySell

Open Orders

Date	Side	Qty (BNB)	Limit (USDT)
No open orders			

For example if user selects BNB/USDT detail, this screen will appear. Trading volume, limit and currently open orders can be seen from this screen.

[Logout](#)[Create Issue](#)

Ali Buğra Şahin

Order submitted.

BNB/USDT

Market Price: 570.0
24-h Trading Volume 14.0

BNB:

USDT:

Limit:

BuySell

Open Orders

Date	Side	Qty (BNB)	Limit (USDT)
2025-05-24 17:10	BUY	1.0000	570.0000

If “BNB:” is filled with desired amount, needed amount of USDT will be shown automatically, as can be seen from this screenshot. Previously entered orders can be seen at the bottom of the page, which there is a buy order currently open that waiting to be fulfilled.

Whenever Buy or Sell order is submitted, it is sent to the database as an open order and it will wait until another order satisfies or partially satisfies the current open order(s). Satisfied orders are removed from database and whole transactions are taken care of, while partial matches will update the database entry accordingly.

[Logout](#)[Create Issue](#)

Ali Buğra Şahin

BNB/USDT

Market Price: 570.0
24-h Trading Volume 1.0

BNB:

USDT:

Limit:

Buy

Sell

Open Orders

Date	Side	Qty (BNB)	Limit (USDT)
No open orders			

Let's open an order to buy 1 BNB. As we can see required amount of USDT is shown after we type the desired BNB amount.

[Logout](#)[Create Issue](#)

Ali Buğra Şahin

Order submitted.

BNB/USDT

Market Price: 570.0
24-h Trading Volume 1.0

BNB:

USDT:

Limit:

Buy

Sell

Open Orders

Date	Side	Qty (BNB)	Limit (USDT)
2025-05-25 12:05	BUY	1.0000	570.0000

After the order is submitted, we can see a confirmation message and currently open orders, which shows our submitted order of 1BNB.

order_id	type	status	quantity	price	creation_time	close_time	user_id	market_id	trade_id
7	36 SELL	FILLED	0	570	2025-05-24 16:41:19.202792	2025-05-24 16:41:19.222096	35	5	21
8	39 SELL	FILLED	0	570	2025-05-24 16:50:45.777493	2025-05-24 16:50:45.797012	35	5	23
9	37 BUY	FILLED	0	570	2025-05-24 16:50:50.935206	2025-05-24 16:50:50.050849	35	5	24
10	40 SELL	FILLED	0	570	2025-05-24 16:50:50.064392	2025-05-24 16:50:50.064392	35	5	24
11	9 BUY	FILLED	0	3200	2025-04-28 17:50:00.000000	2025-05-23 04:11:17.573431	9	9	11
12	21 SELL	FILLED	0	3200	2025-05-23 04:11:17.564615	2025-05-23 04:11:17.585223	35	9	11
13	41 BUY	FILLED	0	580	2025-05-24 16:51:36.021072	2025-05-24 16:51:36.439905	35	5	25
14	5 SELL	FILLED	0	570	2025-04-28 17:54:00.000000	2025-05-23 04:10:28.051107	5	5	12
15	22 BUY	FILLED	0	570	2025-05-23 04:10:28.064312	2025-05-23 04:10:28.059958	35	5	12
16	3 BUY	CANCELLED	500	0.6	2025-04-28 17:57:00.000000	<null>	3	3	5
17	10 BUY	CANCELLED	500	1.08	2025-04-28 17:49:00.000000	<null>	10	10	10
18	6 BUY	CANCELLED	20	150	2025-04-28 17:53:00.000000	<null>	6	6	6
19	8 BUY	CANCELLED	0.12	63000	2025-04-28 17:51:00.000000	<null>	8	8	8
20	23 SELL	FILLED	0	570	2025-05-23 04:30:59.128387	2025-05-23 04:31:09.993075	5	5	13
21	42 SELL	FILLED	0	580	2025-05-24 16:51:45.434951	2025-05-24 16:51:45.453208	35	5	25
22	24 BUY	FILLED	0	570	2025-05-23 04:31:09.985995	2025-05-23 04:31:10.010478	35	5	13
23	25 BUY	FILLED	0	570	2025-05-23 04:32:30.053108	2025-05-23 04:32:30.015281	35	5	14
24	26 SELL	FILLED	0	570	2025-05-23 04:32:30.010679	2025-05-23 04:32:30.026679	5	5	14
25	27 BUY	FILLED	0	570	2025-05-23 16:46:51.137676	2025-05-23 16:47:18.917584	35	5	15
26	44 BUY	OPEN	1	100	2025-05-24 20:37:09.372968	<null>	35	8	<null>
27	28 SELL	FILLED	0	570	2025-05-23 16:47:18.901445	2025-05-23 16:47:18.941094	5	5	15
28	29 BUY	FILLED	0	570	2025-05-24 16:22:25.041352	2025-05-24 16:22:25.054183	35	5	16
29	30 SELL	FILLED	0	570	2025-05-24 16:22:25.043644	2025-05-24 16:22:25.087454	35	5	16
30	31 BUY	FILLED	0	570	2025-05-24 16:22:33.990853	2025-05-24 16:22:40.522234	35	5	17
31	32 BUY	FILLED	0	570	2025-05-24 16:22:33.907064	2025-05-24 16:22:40.536101	35	5	18
32	34 SELL	FILLED	0	570	2025-05-24 16:22:40.517588	2025-05-24 16:22:40.561756	35	5	19
33	43 BUY	FILLED	0	570	2025-05-24 17:10:23.632349	2025-05-25 12:04:05.156873	35	5	27
34	46 SELL	FILLED	0	570	2025-05-25 12:04:05.142526	2025-05-25 12:04:05.172741	35	5	27

Order table before the order is submitted.

order_id	type	status	quantity	price	creation_time	close_time	user_id	market_id	trade_id
7	36 SELL	FILLED	0	570	2025-05-24 16:41:19.202792	2025-05-24 16:41:19.222096	35	5	21
8	39 SELL	FILLED	0	570	2025-05-24 16:50:45.777493	2025-05-24 16:50:45.797012	35	5	23
9	37 BUY	FILLED	0	570	2025-05-24 16:50:50.935206	2025-05-24 16:50:50.050849	35	5	24
10	40 SELL	FILLED	0	570	2025-05-24 16:50:50.064392	2025-05-24 16:50:50.064392	35	5	24
11	9 BUY	FILLED	0	3200	2025-04-28 17:50:00.000000	2025-05-23 04:11:17.573431	9	9	11
12	21 SELL	FILLED	0	3200	2025-05-23 04:11:17.564615	2025-05-23 04:11:17.585223	35	9	11
13	41 BUY	FILLED	0	580	2025-05-24 16:51:36.021072	2025-05-24 16:51:36.439905	35	5	25
14	5 SELL	FILLED	0	570	2025-04-28 17:54:00.000000	2025-05-23 04:10:28.051107	5	5	12
15	22 BUY	FILLED	0	570	2025-05-23 04:10:28.064312	2025-05-23 04:10:28.059958	35	5	12
16	3 BUY	CANCELLED	500	0.6	2025-04-28 17:57:00.000000	<null>	3	3	5
17	10 BUY	CANCELLED	500	1.08	2025-04-28 17:49:00.000000	<null>	10	10	10
18	6 BUY	CANCELLED	20	150	2025-04-28 17:53:00.000000	<null>	6	6	6
19	8 BUY	CANCELLED	0.12	63000	2025-04-28 17:51:00.000000	<null>	8	8	8
20	23 SELL	FILLED	0	570	2025-05-23 04:30:59.128387	2025-05-23 04:31:09.993075	5	5	13
21	42 SELL	FILLED	0	580	2025-05-24 16:51:45.434951	2025-05-24 16:51:45.453208	35	5	25
22	24 BUY	FILLED	0	570	2025-05-23 04:31:09.985995	2025-05-23 04:31:10.010478	35	5	13
23	25 BUY	FILLED	0	570	2025-05-23 04:32:30.053108	2025-05-23 04:32:30.015281	35	5	14
24	26 SELL	FILLED	0	570	2025-05-23 04:32:30.010679	2025-05-23 04:32:30.026679	5	5	14
25	27 BUY	FILLED	0	570	2025-05-23 16:46:51.137676	2025-05-23 16:47:18.917584	35	5	15
26	44 BUY	OPEN	1	100	2025-05-24 20:37:09.372968	<null>	35	8	<null>
27	28 SELL	FILLED	0	570	2025-05-23 16:47:18.901445	2025-05-23 16:47:18.941094	5	5	15
28	29 BUY	FILLED	0	570	2025-05-24 16:22:25.041352	2025-05-24 16:22:25.054183	35	5	16
29	30 SELL	FILLED	0	570	2025-05-24 16:22:25.043644	2025-05-24 16:22:25.087454	35	5	16
30	31 BUY	FILLED	0	570	2025-05-24 16:22:33.990853	2025-05-24 16:22:40.522234	35	5	17
31	32 BUY	FILLED	0	570	2025-05-24 16:22:33.907064	2025-05-24 16:22:40.536101	35	5	18
32	34 SELL	FILLED	0	570	2025-05-24 16:22:40.517588	2025-05-24 16:22:40.561756	35	5	19
33	43 BUY	FILLED	0	570	2025-05-24 17:10:23.632349	2025-05-25 12:04:05.156873	35	5	27
34	46 SELL	FILLED	0	570	2025-05-25 12:04:05.142526	2025-05-25 12:04:05.172741	35	5	27
35	47 BUY	OPEN	1	570	2025-05-25 12:05:06.713120	<null>	35	5	<null>

After the order is placed and confirmation message is shown, it's added to the database as an open order.

[Logout](#)[Create Issue](#)

Ali Buğra Şahin

BNB/USDT

Market Price: 570.0
24-h Trading Volume 1.0

BNB:

USDT:

Limit:

Buy

Sell

Open Orders

Date	Side	Qty (BNB)	Limit (USDT)
No open orders			

Let's say someone wants to sell their 1BNB which will match our previous open order.

[Logout](#)[Create Issue](#)

Ali Buğra Şahin

Order submitted.

BNB/USDT

Market Price: 570.0
24-h Trading Volume 2.0

BNB:

USDT:

Limit:

Buy

Sell

Open Orders

Date	Side	Qty (BNB)	Limit (USDT)
No open orders			

After user clicks "sell" the order will be sent to the database and confirmation message will be shown.

order_id	type	status	quantity	price	creation_time	close_time	user_id	market_id	trade_id
8	SELL	FILLED	0	570	2025-05-24 16:50:45.777493	2025-05-24 16:50:45.797012	35	5	23
9	BUY	FILLED	0	570	2025-05-24 16:50:28.935265	2025-05-24 16:50:50.050869	35	5	24
10	SELL	FILLED	0	570	2025-05-24 16:50:50.045734	2025-05-24 16:50:50.064392	35	5	24
11	BUY	FILLED	0	3200	2025-04-28 17:50:00.000000	2025-05-23 04:11:17.573431	9	9	11
12	SELL	FILLED	0	3200	2025-05-23 04:11:17.564615	2025-05-23 04:11:17.585223	35	5	11
13	BUY	FILLED	0	580	2025-05-24 16:51:36.021672	2025-05-24 16:51:45.439905	35	5	25
14	SELL	FILLED	0	570	2025-04-28 17:54:00.000000	2025-05-23 04:10:28.051107	5	5	12
15	BUY	FILLED	0	570	2025-05-23 04:10:28.041312	2025-05-23 04:10:28.059950	35	5	12
16	BUY	CANCELLED	500	0.6	2025-04-28 17:57:09.000000	<null>	3	3	3
17	BUY	CANCELLED	500	1.08	2025-04-28 17:49:09.000000	<null>	10	10	10
18	BUY	CANCELLED	20	150	2025-04-28 17:53:09.000000	<null>	6	6	6
19	BUY	CANCELLED	0.12	65000	2025-04-28 17:51:09.000000	<null>	8	8	8
20	SELL	FILLED	0	570	2025-05-23 04:30:59.128387	2025-05-23 04:31:09.993075	5	5	13
21	SELL	FILLED	0	580	2025-05-24 16:51:45.434991	2025-05-24 16:51:45.453208	35	5	25
22	BUY	FILLED	0	570	2025-05-23 04:31:09.985995	2025-05-23 04:31:10.010478	35	5	13
23	BUY	FILLED	0	570	2025-05-23 04:32:30.035108	2025-05-23 04:32:30.015281	35	5	14
24	SELL	FILLED	0	570	2025-05-23 04:32:30.010679	2025-05-23 04:32:30.026679	5	5	14
25	BUY	FILLED	0	570	2025-05-23 16:44:51.137674	2025-05-23 16:47:18.917584	35	5	15
26	BUY	OPEN	1	100	2025-05-24 20:37:09.372968	<null>	35	8	<null>
27	SELL	FILLED	0	570	2025-05-23 16:47:18.901445	2025-05-23 16:47:18.941094	5	5	15
28	BUY	FILLED	0	570	2025-05-24 16:22:25.091352	2025-05-24 16:22:25.054183	35	5	16
29	SELL	FILLED	0	570	2025-05-24 16:22:25.043044	2025-05-24 16:22:25.087454	35	5	16
30	BUY	FILLED	0	570	2025-05-24 16:22:31.890853	2025-05-24 16:22:40.522234	35	5	17
31	BUY	FILLED	0	570	2025-05-24 16:22:33.907804	2025-05-24 16:22:40.550101	35	5	18
32	SELL	FILLED	0	570	2025-05-24 16:22:40.517588	2025-05-24 16:22:40.561756	35	5	19
33	BUY	FILLED	0	570	2025-05-24 17:10:23.632249	2025-05-23 12:04:05.156873	35	5	27
34	SELL	FILLED	0	570	2025-05-25 12:04:05.142526	2025-05-25 12:04:05.172741	35	5	27
35	BUY	OPEN	1	570	2025-05-25 12:05:46.713120	<null>	35	5	<null>

Order table before sell order is placed. We can see the open order of “**BUY 1BNB**” at the bottom of the list.

order_id	type	status	quantity	price	creation_time	close_time	user_id	market_id	trade_id
8	SELL	FILLED	0	570	2025-05-24 16:50:45.777493	2025-05-24 16:50:45.797012	35	5	23
9	BUY	FILLED	0	570	2025-05-24 16:50:28.935265	2025-05-24 16:50:50.050869	35	5	24
10	SELL	FILLED	0	570	2025-05-24 16:50:50.045734	2025-05-24 16:50:50.064392	35	5	24
11	BUY	FILLED	0	3200	2025-04-28 17:50:00.000000	2025-05-23 04:11:17.573431	9	9	11
12	SELL	FILLED	0	3200	2025-05-23 04:11:17.564615	2025-05-23 04:11:17.585223	35	5	11
13	BUY	FILLED	0	580	2025-05-24 16:51:36.021672	2025-05-24 16:51:45.439905	35	5	25
14	SELL	FILLED	0	570	2025-04-28 17:54:00.000000	2025-05-23 04:10:28.051107	5	5	12
15	BUY	FILLED	0	570	2025-05-23 04:10:28.041312	2025-05-23 04:10:28.059950	35	5	12
16	BUY	CANCELLED	500	0.6	2025-04-28 17:57:09.000000	<null>	3	3	3
17	BUY	CANCELLED	500	1.08	2025-04-28 17:49:09.000000	<null>	10	10	10
18	BUY	CANCELLED	20	150	2025-04-28 17:53:09.000000	<null>	6	6	6
19	BUY	CANCELLED	0.12	65000	2025-04-28 17:51:09.000000	<null>	8	8	8
20	SELL	FILLED	0	570	2025-05-23 04:30:59.128387	2025-05-23 04:31:09.993075	5	5	13
21	SELL	FILLED	0	580	2025-05-24 16:51:45.434991	2025-05-24 16:51:45.453208	35	5	25
22	BUY	FILLED	0	570	2025-05-23 04:31:09.985995	2025-05-23 04:31:10.010478	35	5	13
23	BUY	FILLED	0	570	2025-05-23 04:32:30.035108	2025-05-23 04:32:30.015281	35	5	14
24	SELL	FILLED	0	570	2025-05-23 04:32:30.010679	2025-05-23 04:32:30.026679	5	5	14
25	BUY	FILLED	0	570	2025-05-23 16:44:51.137674	2025-05-23 16:47:18.917584	35	5	15
26	BUY	OPEN	1	100	2025-05-24 20:37:09.372968	<null>	35	8	<null>
27	SELL	FILLED	0	570	2025-05-23 16:47:18.901445	2025-05-23 16:47:18.941094	5	5	15
28	BUY	FILLED	0	570	2025-05-24 16:22:25.091352	2025-05-24 16:22:25.054183	35	5	16
29	SELL	FILLED	0	570	2025-05-24 16:22:25.043044	2025-05-24 16:22:25.087454	35	5	16
30	BUY	FILLED	0	570	2025-05-24 16:22:31.890853	2025-05-24 16:22:40.522234	35	5	17
31	BUY	FILLED	0	570	2025-05-24 16:22:33.907804	2025-05-24 16:22:40.550101	35	5	18
32	SELL	FILLED	0	570	2025-05-24 16:22:40.517588	2025-05-24 16:22:40.561756	35	5	19
33	BUY	FILLED	0	570	2025-05-24 17:10:23.632249	2025-05-25 12:04:05.156873	35	5	27
34	SELL	FILLED	0	570	2025-05-25 12:04:05.142526	2025-05-25 12:04:05.172741	35	5	27
35	BUY	FILLED	0	570	2025-05-25 12:05:46.713120	2025-05-25 12:09:37.151561	35	5	28
36	SELL	FILLED	0	570	2025-05-25 12:09:37.147081	2025-05-25 12:09:37.160997	35	5	28

After the “**SELL 1BNB**” order is placed, we can see that it matched with our “**BUY 1BNB**” order and the status has changed from OPEN to FILLED.

trade_id	price	quantity	trade_time	market_id
4	0.52	1500	2025-04-28 17:56:00.000000	4
5	570	10	2025-04-28 17:54:00.000000	5
6	150	20	2025-04-28 17:53:00.000000	6
7	44	50	2025-04-28 17:52:00.000000	7
8	63000	0.12	2025-04-28 17:51:00.000000	8
9	3200	0.8	2025-05-21 22:15:38.417503	9
10	1.08	500	2025-04-28 17:49:00.000000	10
11	3200	1	2025-05-23 04:11:17.572212	9
12	570	1	2025-05-23 04:16:28.050079	5
13	570	1	2025-05-23 04:31:09.990611	5
14	570	1	2025-05-23 04:32:30.014334	5
15	570	1	2025-05-23 16:47:18.909833	5
16	570	1	2025-05-24 16:22:25.048455	5
17	570	1	2025-05-24 16:22:40.521151	5
18	570	1	2025-05-24 16:22:40.534426	5
19	570	3	2025-05-24 16:22:40.549420	5
20	570	1	2025-05-24 16:41:13.672770	5
21	570	1	2025-05-24 16:41:19.207151	5
22	570	4	2025-05-24 16:50:45.781493	5
23	570	1	2025-05-24 16:50:50.049523	5
24	580	1	2025-05-24 16:51:45.438837	5
25	570	1	2025-05-25 12:04:05.154275	5

Trade table, which keeps track of all transactions done, before the 1BNB BUY/SELL transaction has happened.

trade_id	price	quantity	trade_time	market_id
4	0.52	1500	2025-04-28 17:56:00.000000	4
5	570	10	2025-04-28 17:54:00.000000	5
6	150	20	2025-04-28 17:53:00.000000	6
7	44	50	2025-04-28 17:52:00.000000	7
8	63000	0.12	2025-04-28 17:51:00.000000	8
9	3200	0.8	2025-05-21 22:15:38.417503	9
10	1.08	500	2025-04-28 17:49:00.000000	10
11	3200	1	2025-05-23 04:11:17.572212	9
12	570	1	2025-05-23 04:16:28.050079	5
13	570	1	2025-05-23 04:31:09.990611	5
14	570	1	2025-05-23 04:32:30.014334	5
15	570	1	2025-05-23 16:47:18.909833	5
16	570	1	2025-05-24 16:22:25.048455	5
17	570	1	2025-05-24 16:22:40.521151	5
18	570	1	2025-05-24 16:22:40.534426	5
19	570	3	2025-05-24 16:22:40.549420	5
20	570	1	2025-05-24 16:41:13.672770	5
21	570	1	2025-05-24 16:41:19.207151	5
22	570	4	2025-05-24 16:50:45.781493	5
23	570	1	2025-05-24 16:50:50.049523	5
24	580	1	2025-05-24 16:51:45.438837	5
25	570	1	2025-05-25 12:04:05.154275	5
26	570	1	2025-05-25 12:09:37.150610	5

After the 1BNB BUY/SELL transaction successfully matches, it's added to the Trade table with its details such as trade ID, trade price (in USDT) and amount of symbol that has been traded.